Level 1: Hybrid Environment and Ingestion (Code-First)

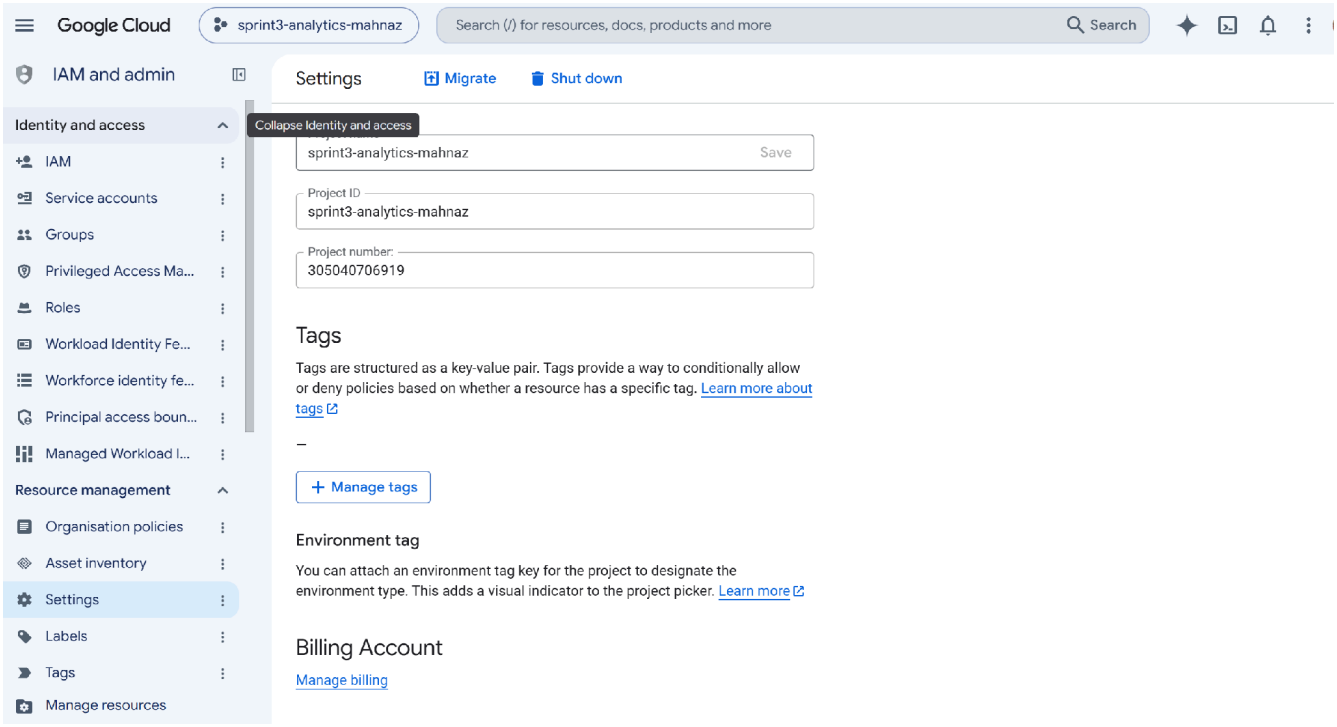
Exercise 1: Data Architecture (Logical vs. Physical)

You will physically implement the logical architecture by creating three Datasets within the same project to centralize billing and permissions.

Create a **New Project** in Google Cloud called:

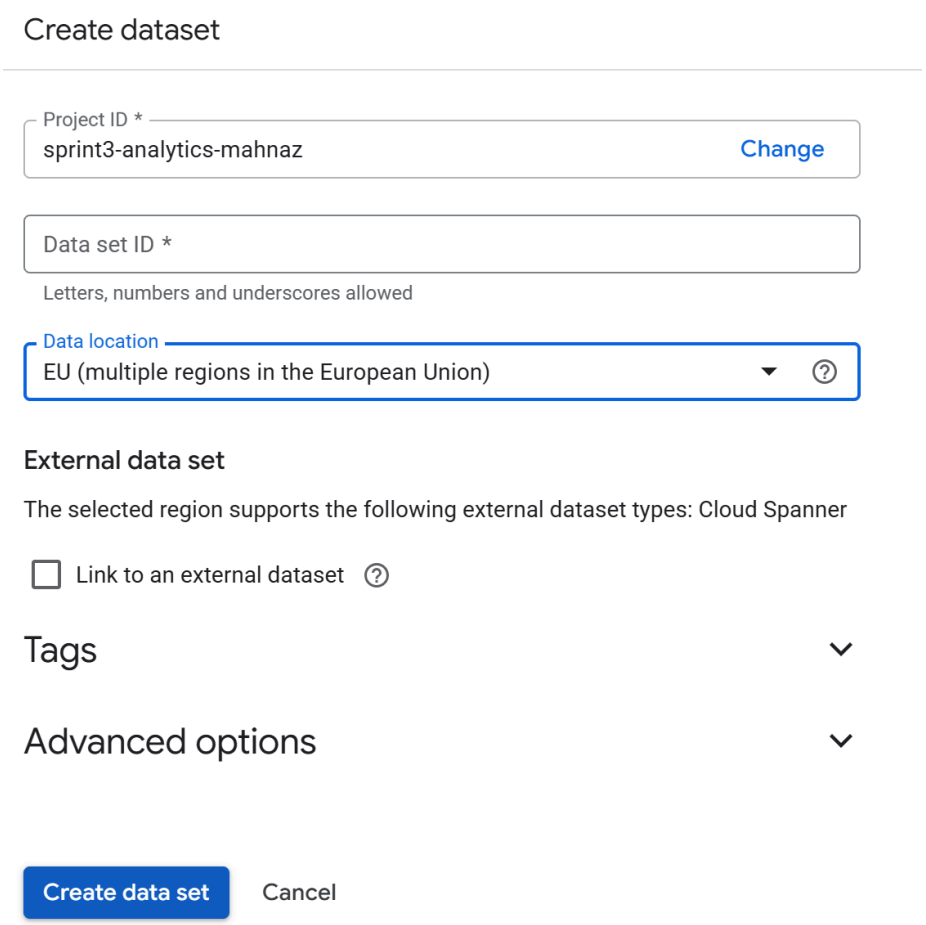
sprint3-analytics-[el-teu-nom].

> *You can create it through the UI, but if you do it through the Console (gcloud CLI) it has a plus.*

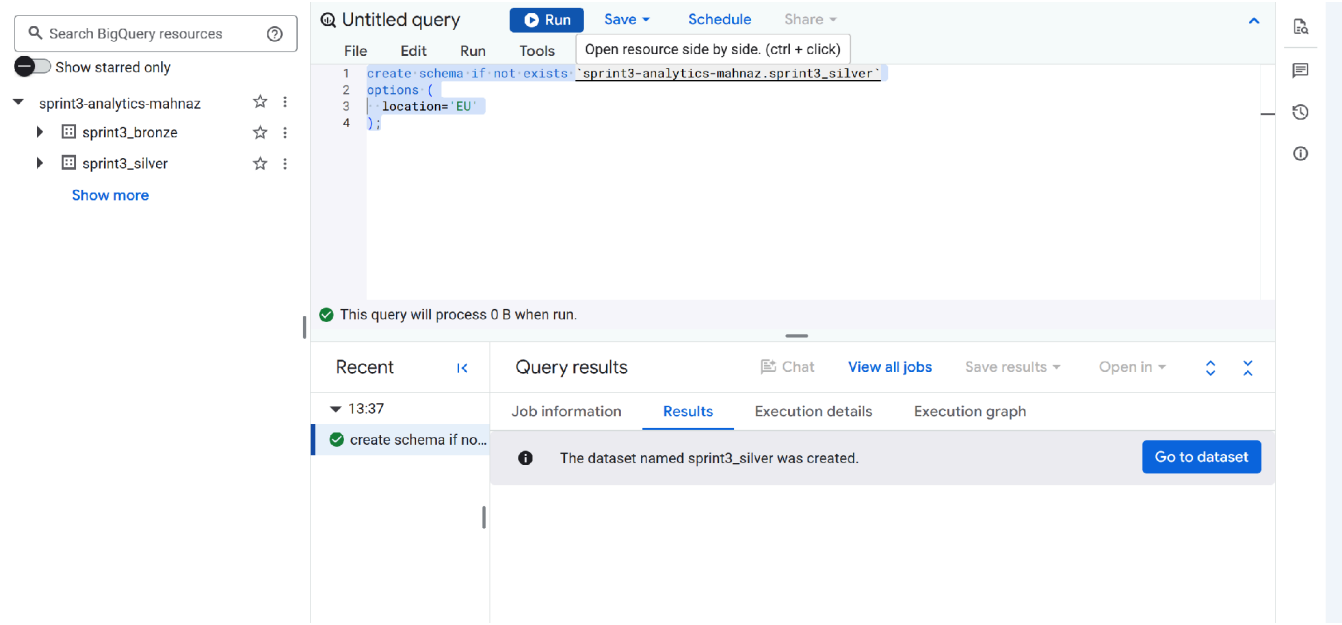


Physical Dataset sprint3_bronze → Bronze Logical Layer (Raw Data)

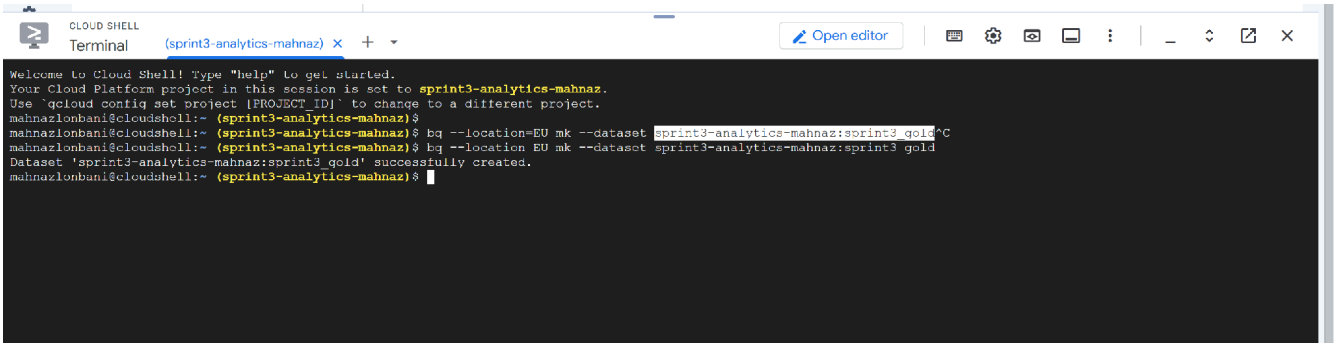
Function: Raw data as it arrives from the source.
Method: Use the BigQuery **Graphical Interface (UI) (click-through).**



Physical Dataset sprint3_silver → Silver Logical Layer (Clean Data)
unction: Clean, typed and deduplicated data.
Method: Use SQL code (CREATE SCHEMA).



Physical Dataset sprint3_gold → Gold Logical Layer (Business Data)
Function: Aggregated data ready for reports and dashboards.
Method: Use Cloud Shell (bq Command Line).



Exercise 2: Ingestion in Bronze Layer (DDL Connection)

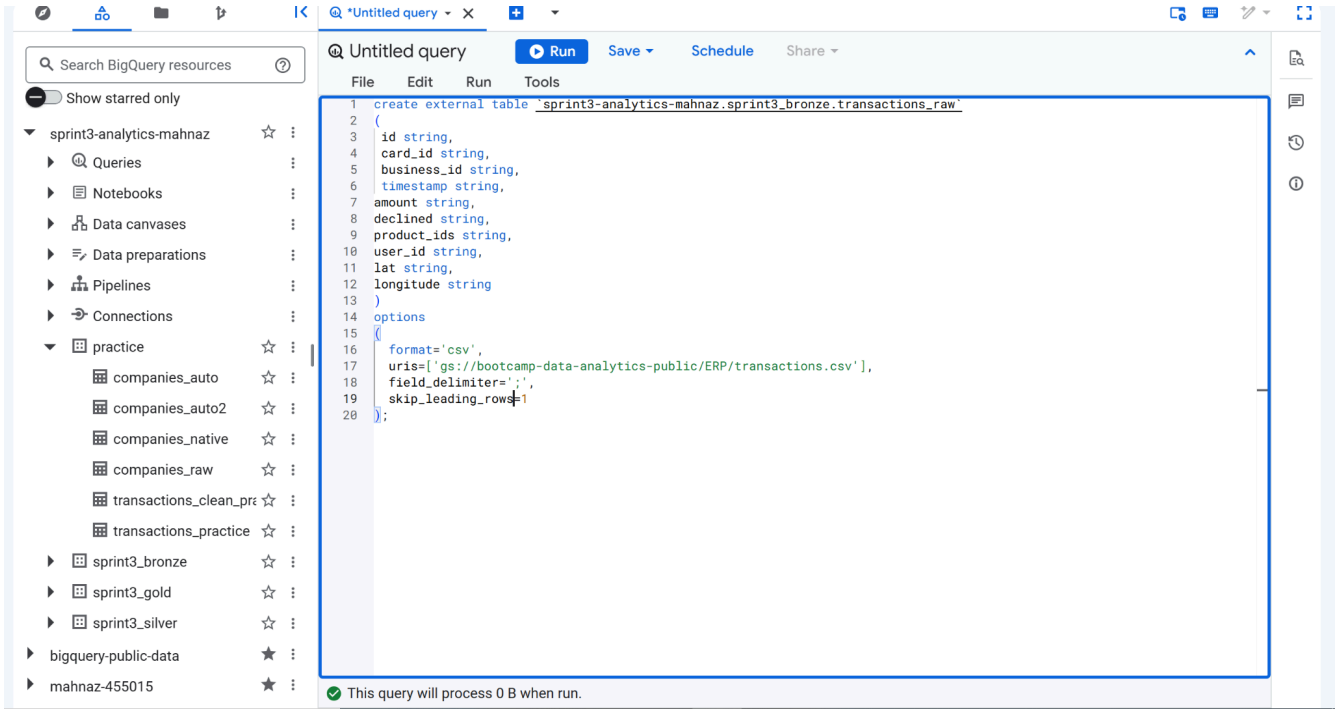
A professional analyst automates the tasks. Now that you have your architecture ready, we will populate the first layer (Bronze) without moving the original data, creating External Tables that point to the Data Lake.

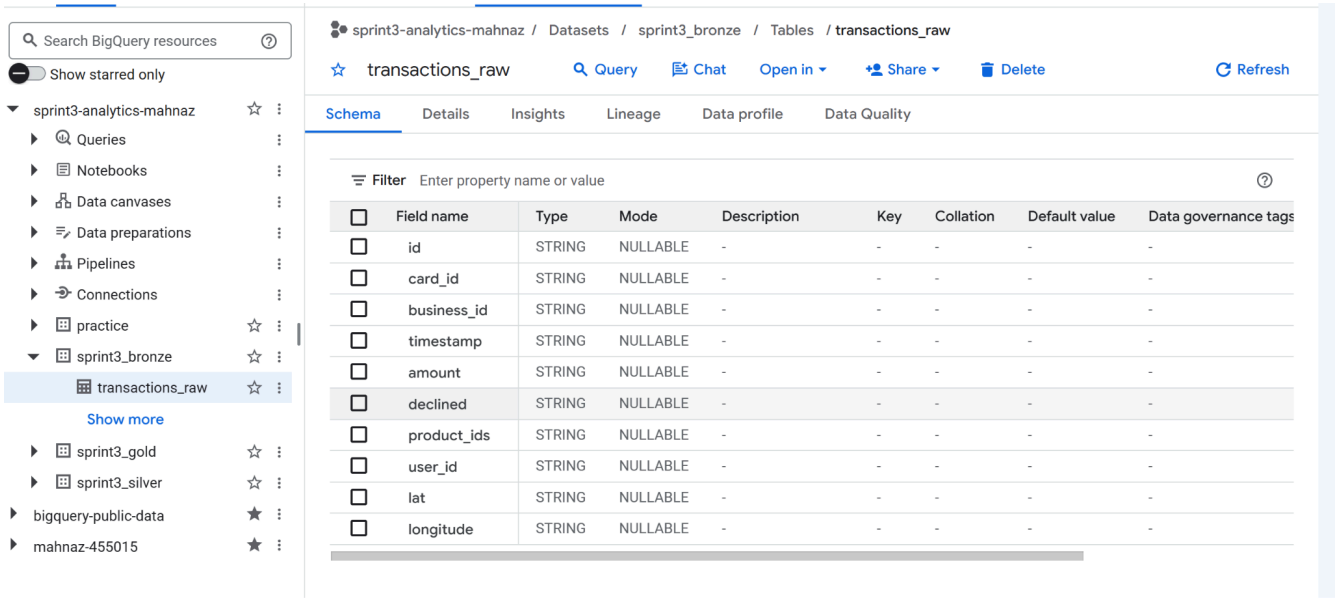
Task:

Write and execute the statements CREATE EXTERNAL TABLEto connect the following files to the sprint3_bronze dataset . Pay close attention to the "Technical Notes", as not all files have the same format.

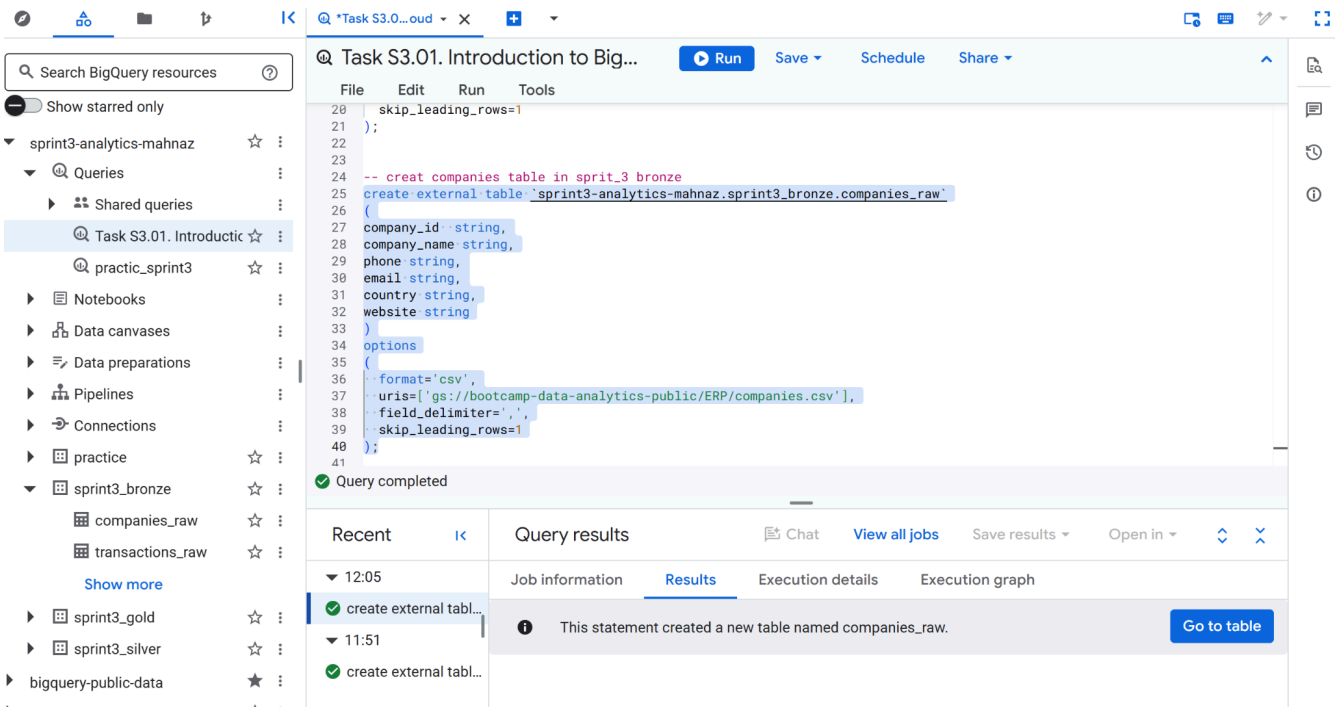
`transactions_raw` table:

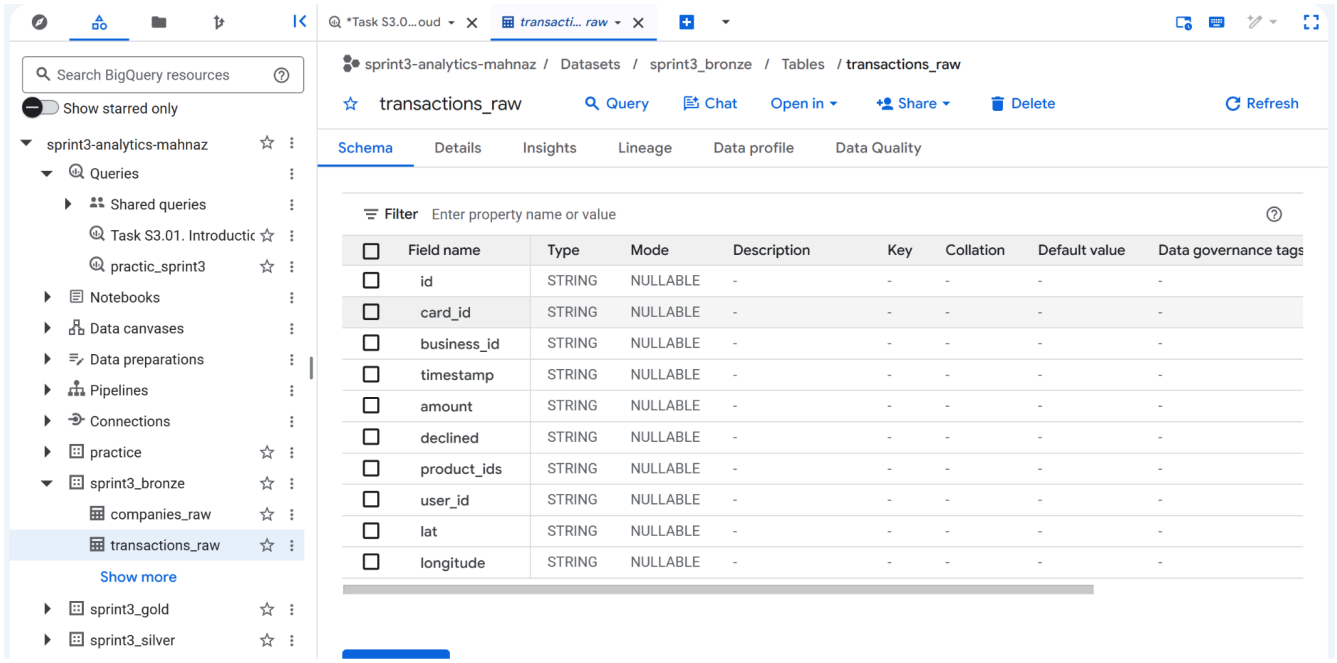
```
create external table `sprint3-analytics-mahnaz.sprint3_bronze.transactions_raw`
(
 id string,
 card_id string,
 business_id string,
 timestamp string,
amount string,
declined string,
product_ids string,
user_id string,
lat string,
longitude string
)
options
(
  format='csv',
  uris=['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],
  field_delimiter=';',
  skip_leading_rows=1
);
```


*Untitled query
Search BigQuery resources
Show starred only
sprint3-analytics-mahnaz
Queries
Notebooks
Data canvases
Data preparations
Pipelines
Connections
practice
companies_auto
companies_auto2
companies_native
companies_raw
transactions_clean_pra
transactions_practice
sprint3_bronze
sprint3_gold
sprint3_silver
bigquery-public-data
mahnaz-455015
Untitled query
Run
Save
Schedule
Share
File
Edit
Run
Tools
create external table `sprint3-analytics-mahnaz.sprint3_bronze.transactions_raw`
(
id string,
card_id string,
business_id string,
timestamp string,
amount string,
declined string,
product_ids string,
user_id string,
lat string,
longitude string
)
options
(
format='csv',
uris=['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],
field_delimiter=';',
skip_leading_rows=1
);
This query will process 0 B when run.


Search BigQuery resources
Show starred only
sprint3-analytics-mahnaz
Queries
Notebooks
Data canvases
Data preparations
Pipelines
Connections
practice
sprint3_bronze
transactions_raw
Show more
sprint3_gold
sprint3_silver
bigquery-public-data
mahnaz-455015
sprint3-analytics-mahnaz / Datasets / sprint3_bronze / Tables / transactions_raw
transactions_raw
Query
Chat
Open in
Share
Delete
Refresh
Schema
Details
Insights
Lineage
Data profile
Data Quality
Filter
Enter property name or value
Field name
Type
Mode
Description
Key
Collation
Default value
Data governance tags
id STRING NULLABLE
card_id STRING NULLABLE
business_id STRING NULLABLE
timestamp STRING NULLABLE
amount STRING NULLABLE
declined STRING NULLABLE
product_ids STRING NULLABLE
user_id STRING NULLABLE
lat STRING NULLABLE
longitude STRING NULLABLE

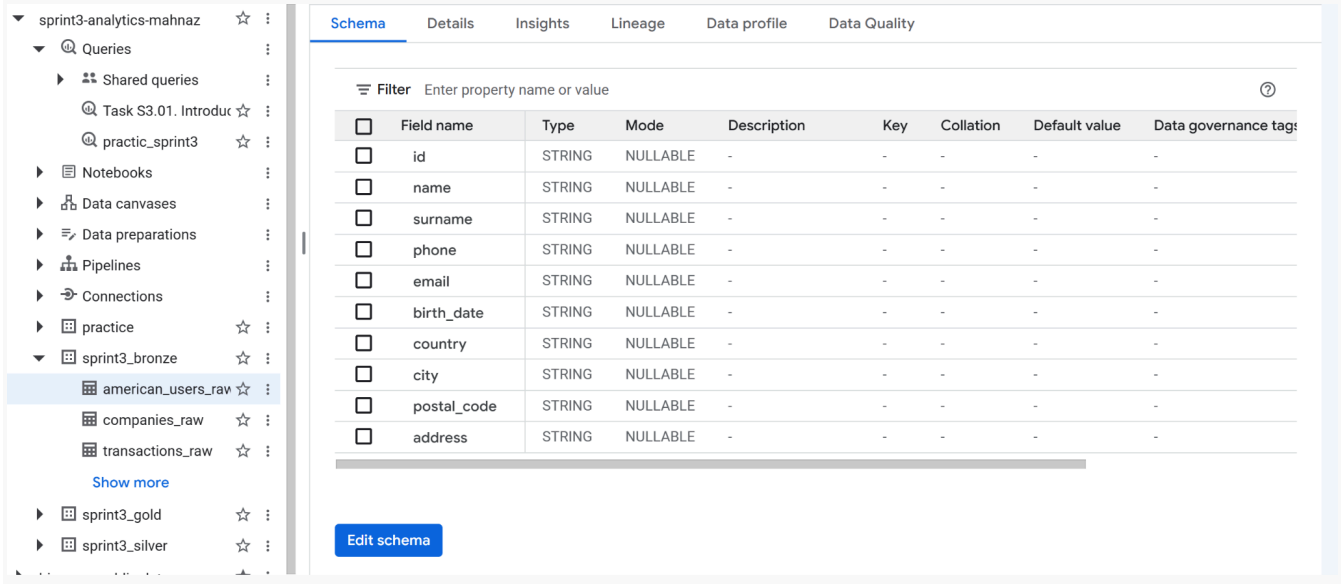
companies_raw table:



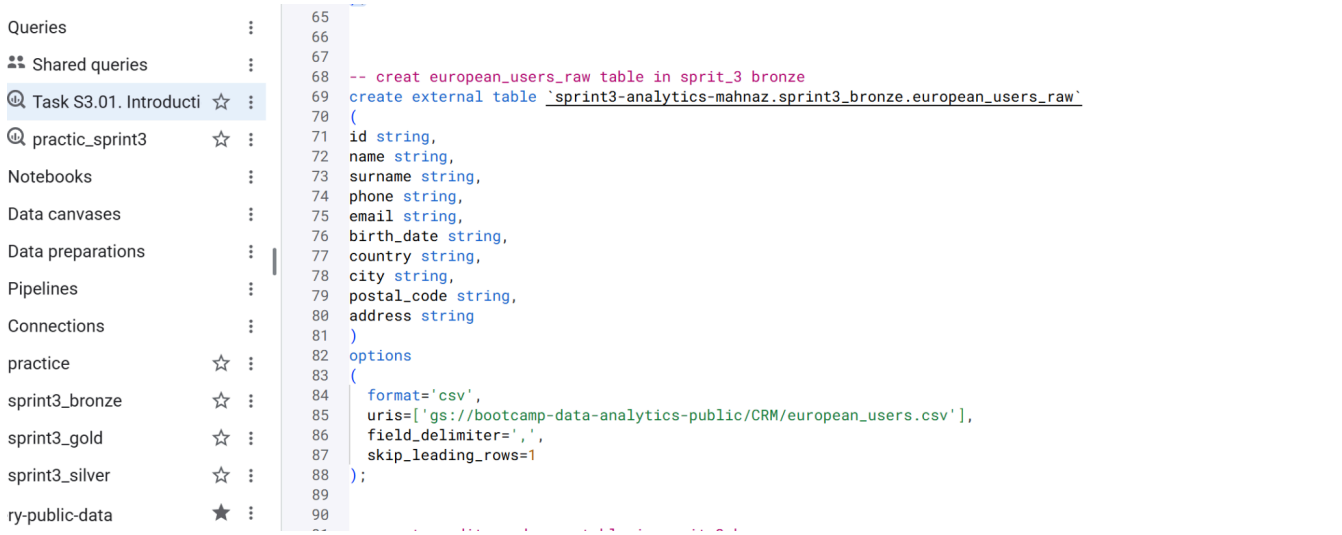


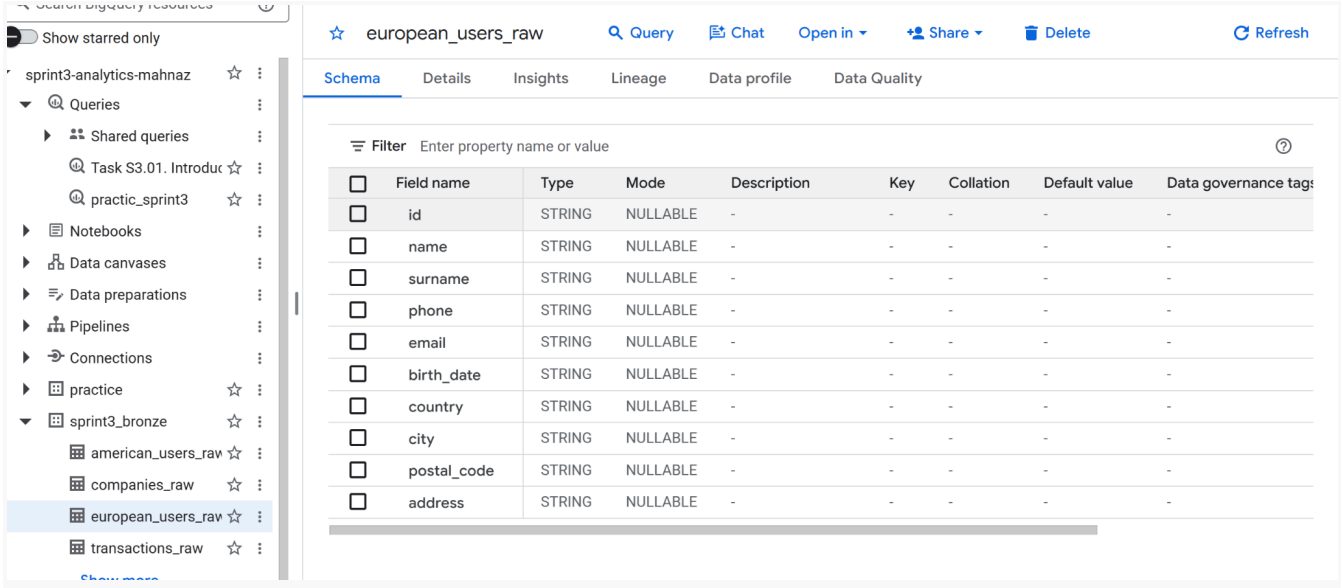
american_users_raw table:



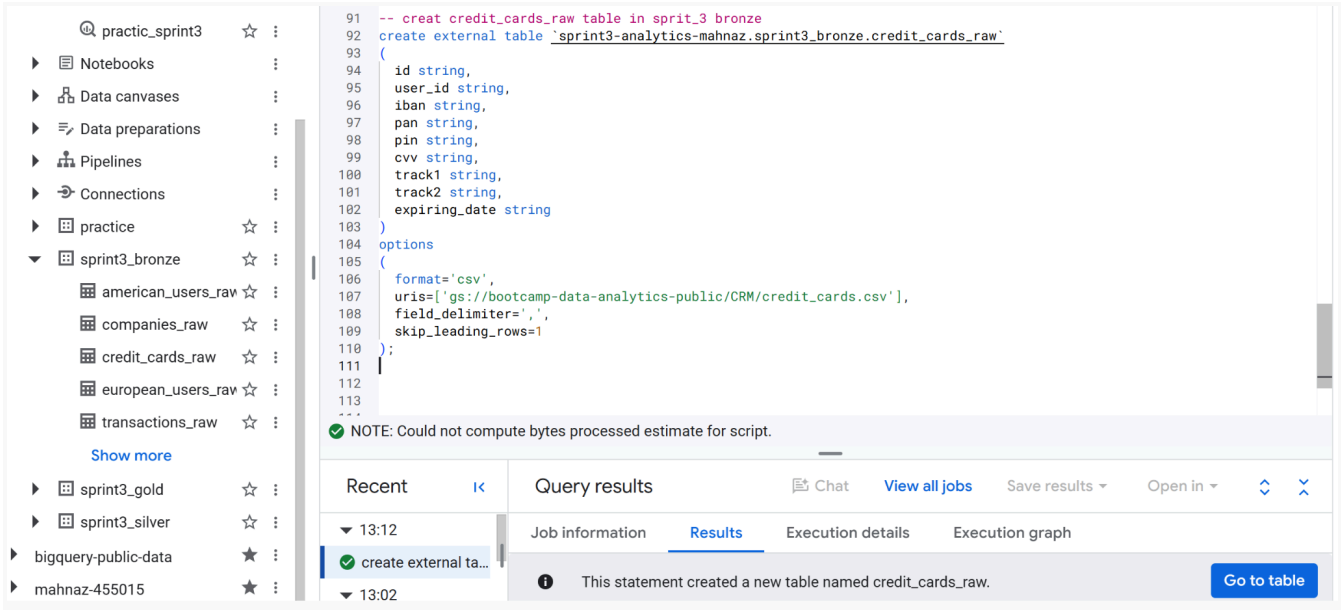


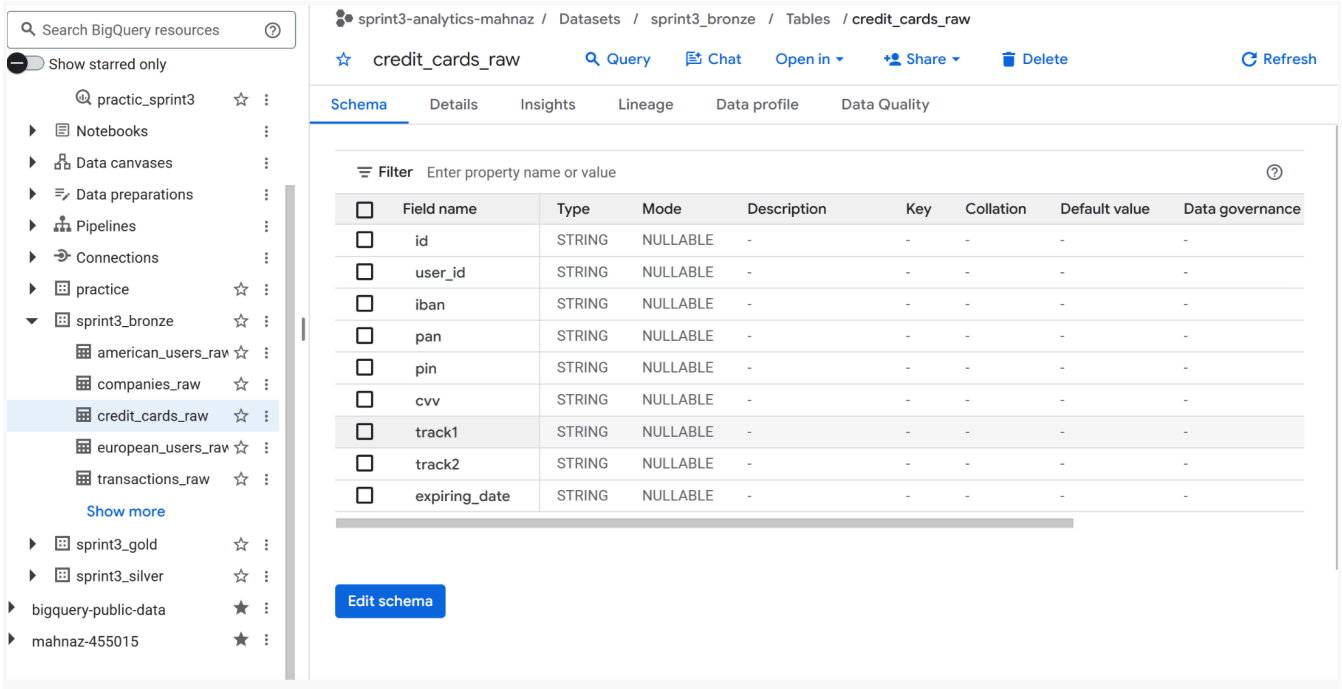
european _users_raw table:





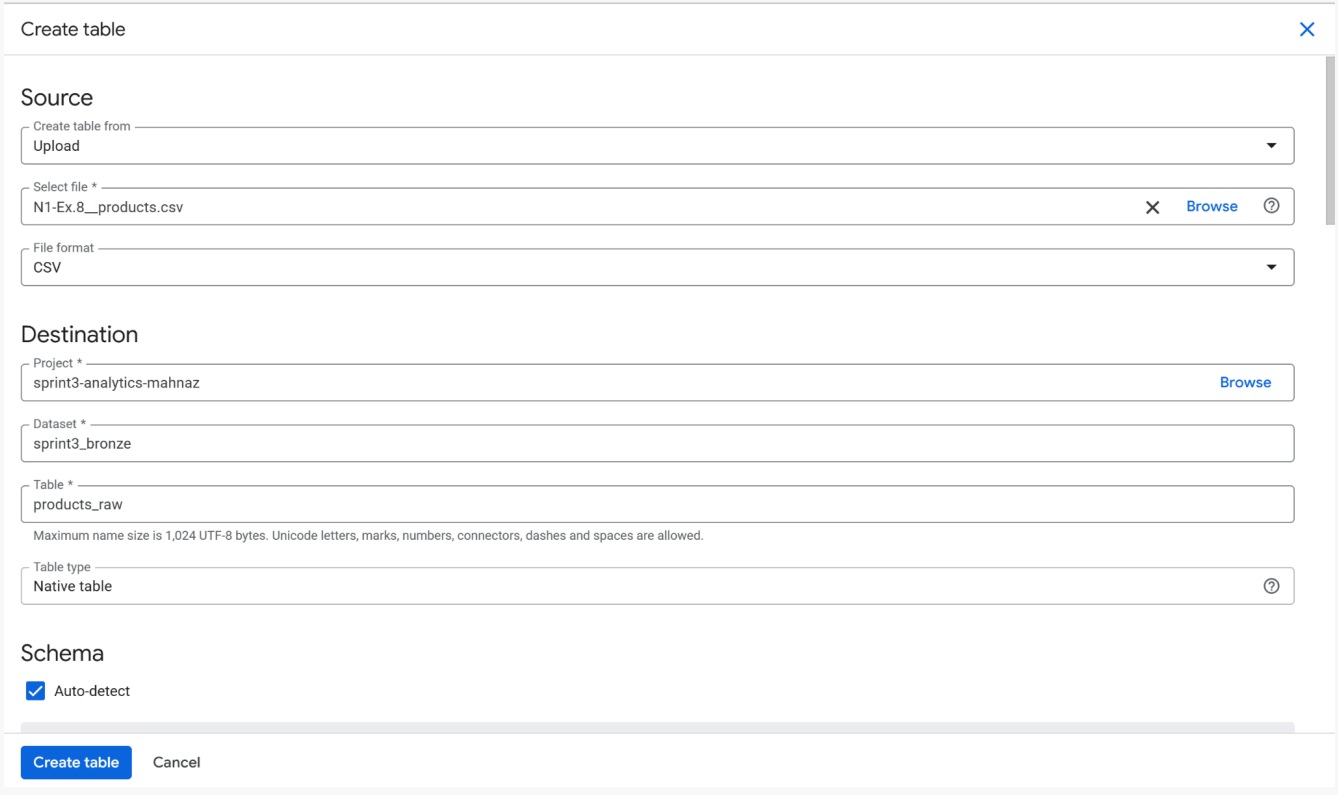
credit_cards_raw table:

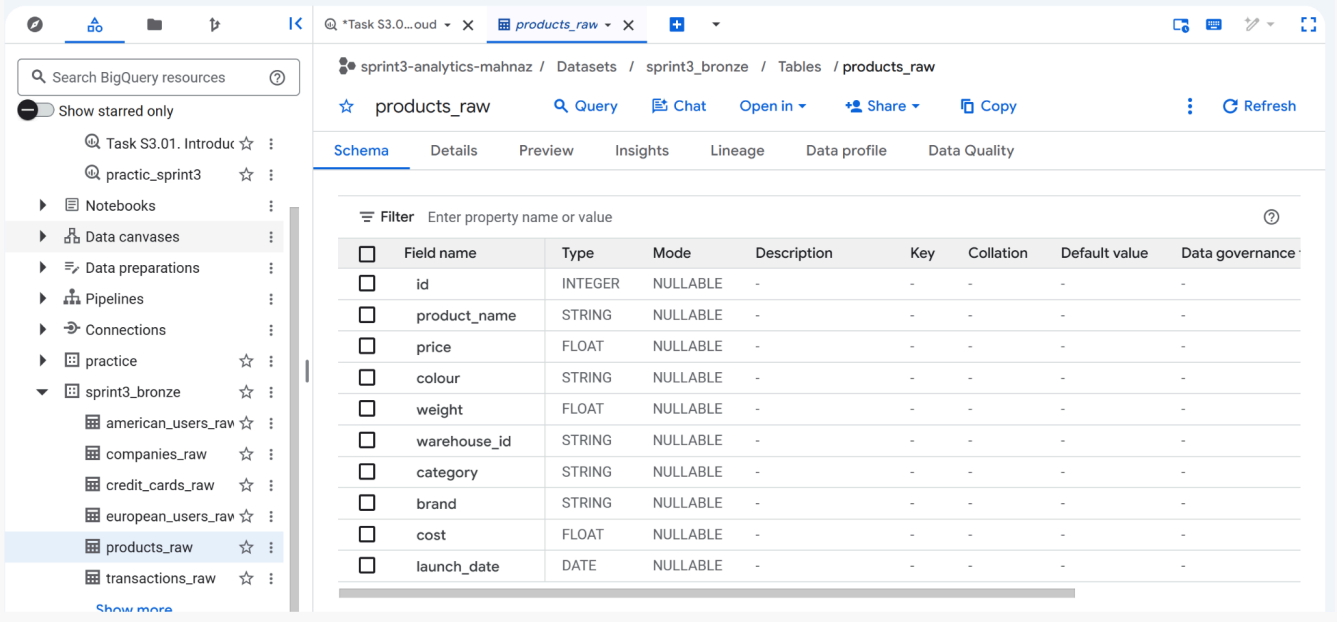




Exercise 3: Uploading Local Data

The Products catalog is missing . This information is not in the Data Lake, use the *products.csv* file from sprint 2.

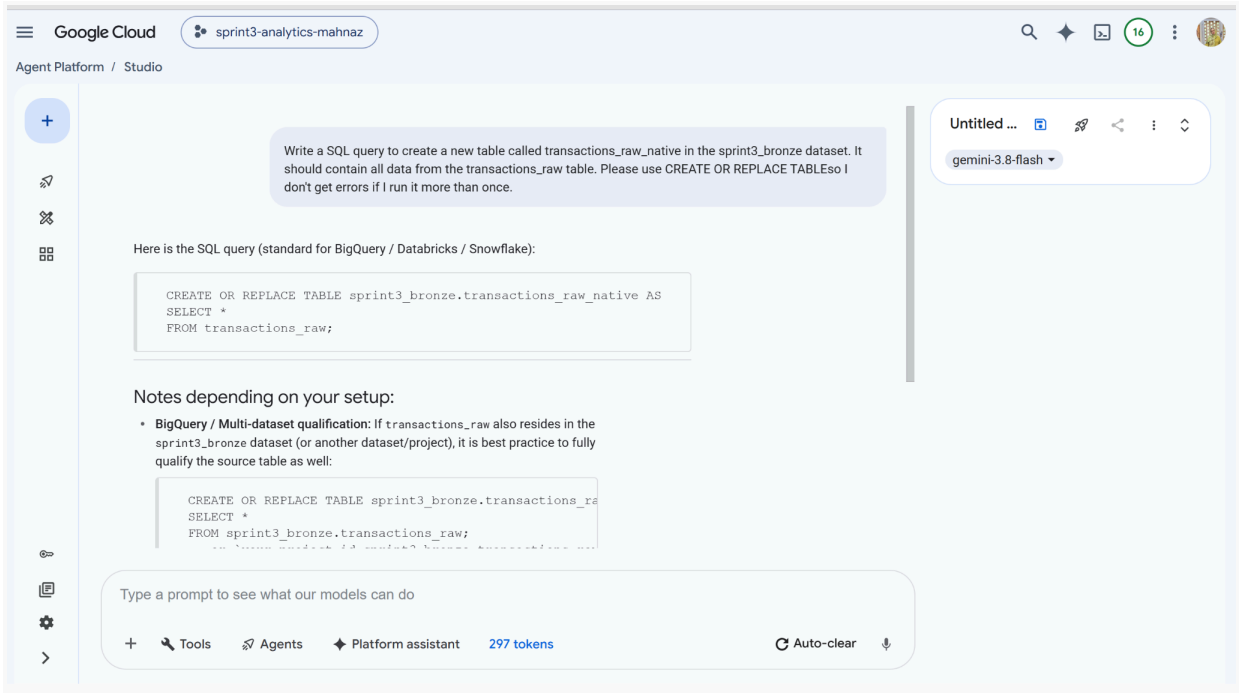


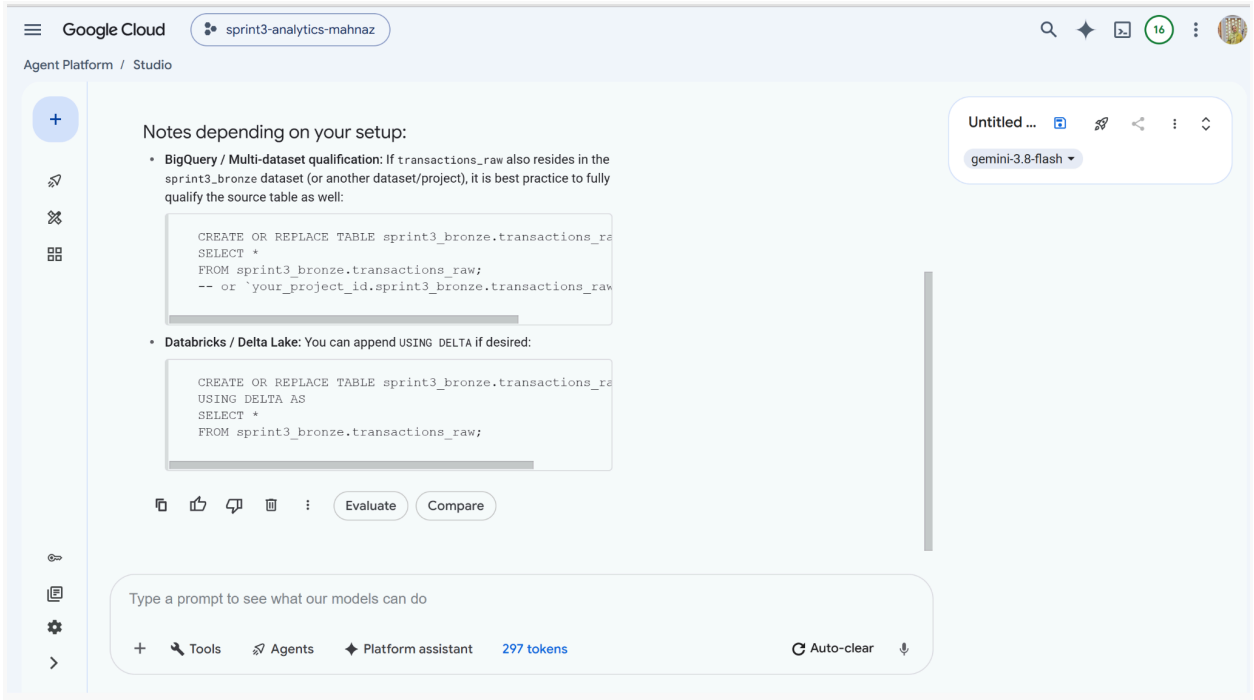


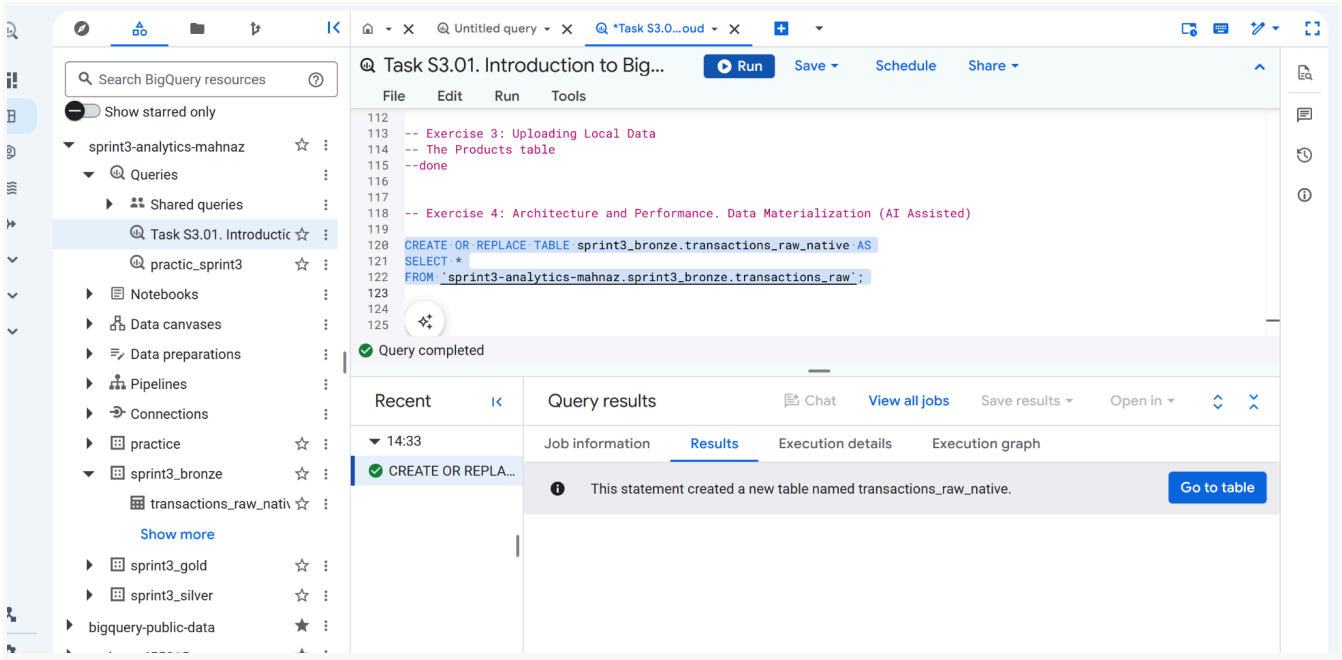
Exercise 4: Architecture and Performance. Data Materialization (AI Assisted)

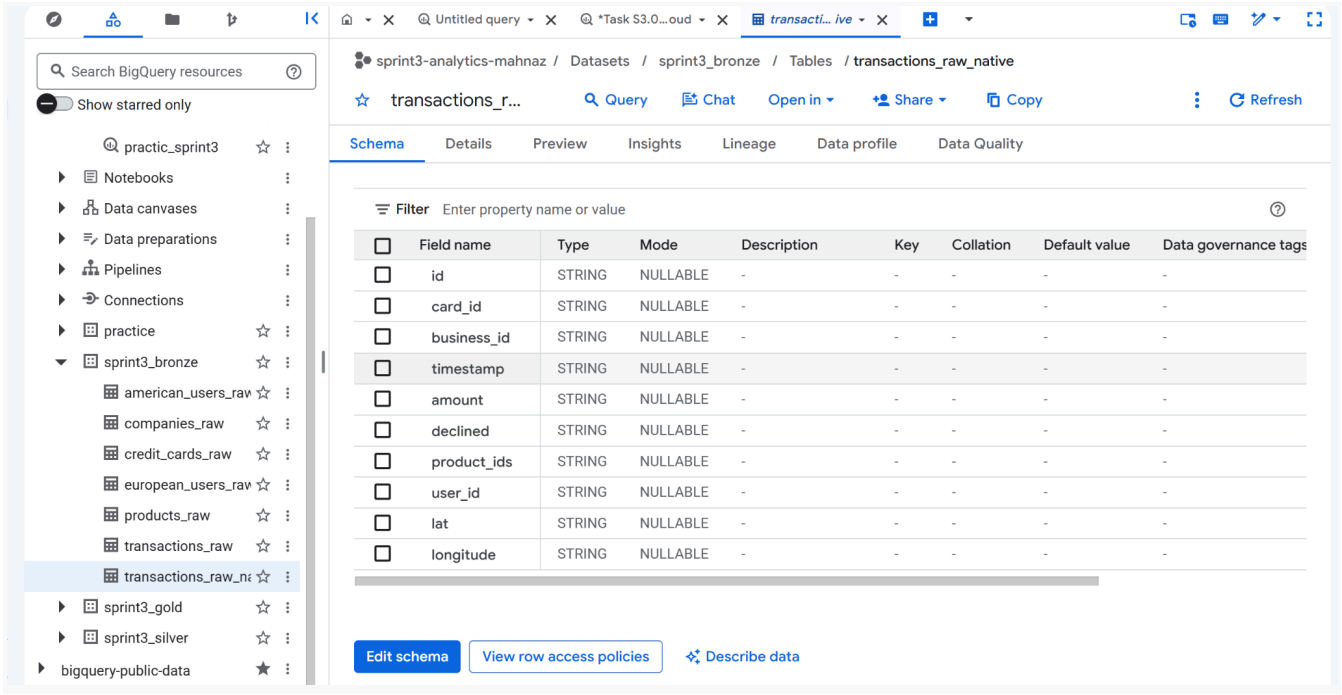
a)Use the AI Assistant (Gemini) built into BigQuery (or the side chat) to generate the statement.

Prompt: "Write a SQL query to create a new table called transactions_raw_native in the sprint3_bronze dataset. It should contain all data from the transactions_raw table. Please use CREATE OR REPLACE TABLEso I don't get errors if I run it more than once."






Search BigQuery resources
Show starred only
sprint3-analytics-mahnaz
Queries
Shared queries
Task S3.01. Introductic
practic_sprint3
Notebooks
Data canvases
Data preparations
Pipelines
Connections
practice
sprint3_bronze
transactions_raw_nati
Show more
sprint3_gold
sprint3_silver
bigquery-public-data
Untitled query
*Task S3.0...oud
Task S3.01. Introduction to Big...
Run
Save
Schedule
Share
File
Edit
Run
Tools
-- Exercise 3: Uploading Local Data
-- The Products table
--done
-- Exercise 4: Architecture and Performance. Data Materialization (AI Assisted)
CREATE OR REPLACE TABLE sprint3_bronze.transactions_raw_native AS
SELECT *
FROM `sprint3-analytics-mahnaz.sprint3_bronze.transactions_raw`;
Query completed
Recent
14:33
CREATE OR REPLA...
Query results
Chat
View all jobs
Save results
Open in
Job information
Results
Execution details
Execution graph
This statement created a new table named transactions_raw_native.
Go to table

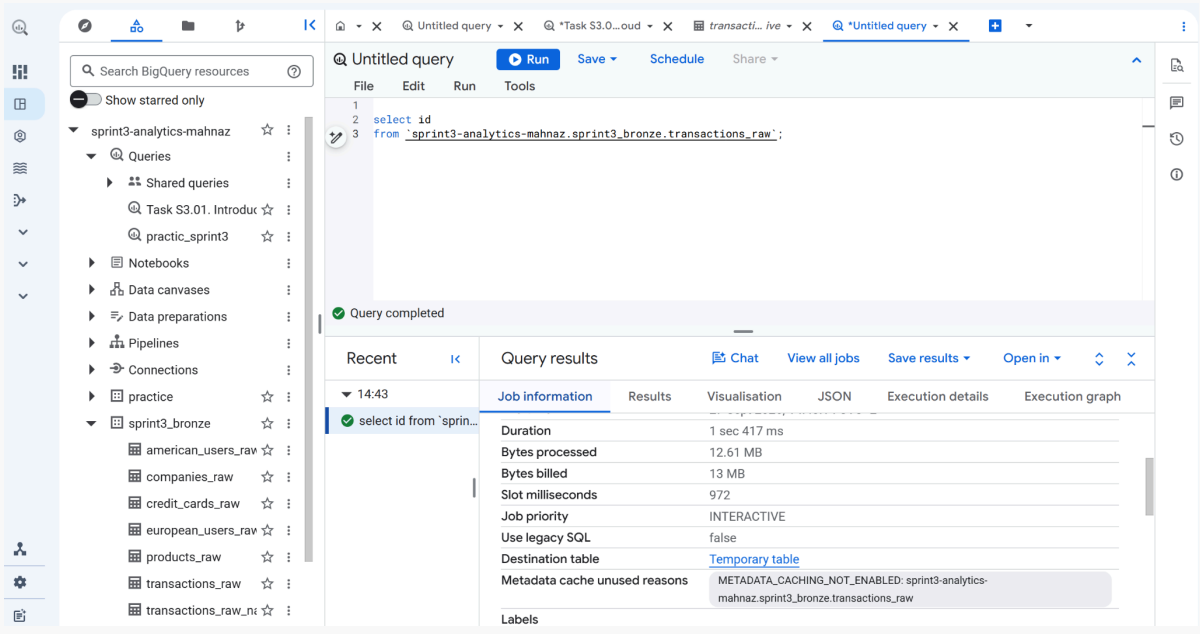

Search BigQuery resources
Show starred only
practic_sprint3
Notebooks
Data canvases
Data preparations
Pipelines
Connections
practice
sprint3_bronze
american_users_raw
companies_raw
credit_cards_raw
european_users_raw
products_raw
transactions_raw
transactions_raw_na
sprint3_gold
sprint3_silver
bigquery-public-data
Untitled query
*Task S3.0...oud
transacti... ive
sprint3-analytics-mahnaz / Datasets / sprint3_bronze / Tables / transactions_raw_native
transactions_r...
Query
Chat
Open in
Share
Copy
Refresh
Schema
Details
Preview
Insights
Lineage
Data profile
Data Quality
Filter
Enter property name or value
Field name
Type
Mode
Description
Key
Collation
Default value
Data governance tags
id STRING NULLABLE
card_id STRING NULLABLE
business_id STRING NULLABLE
timestamp STRING NULLABLE
amount STRING NULLABLE
declined STRING NULLABLE
product_ids STRING NULLABLE
user_id STRING NULLABLE
lat STRING NULLABLE
longitude STRING NULLABLE
Edit schema
View row access policies
Describe data

b) **Cost Audit**

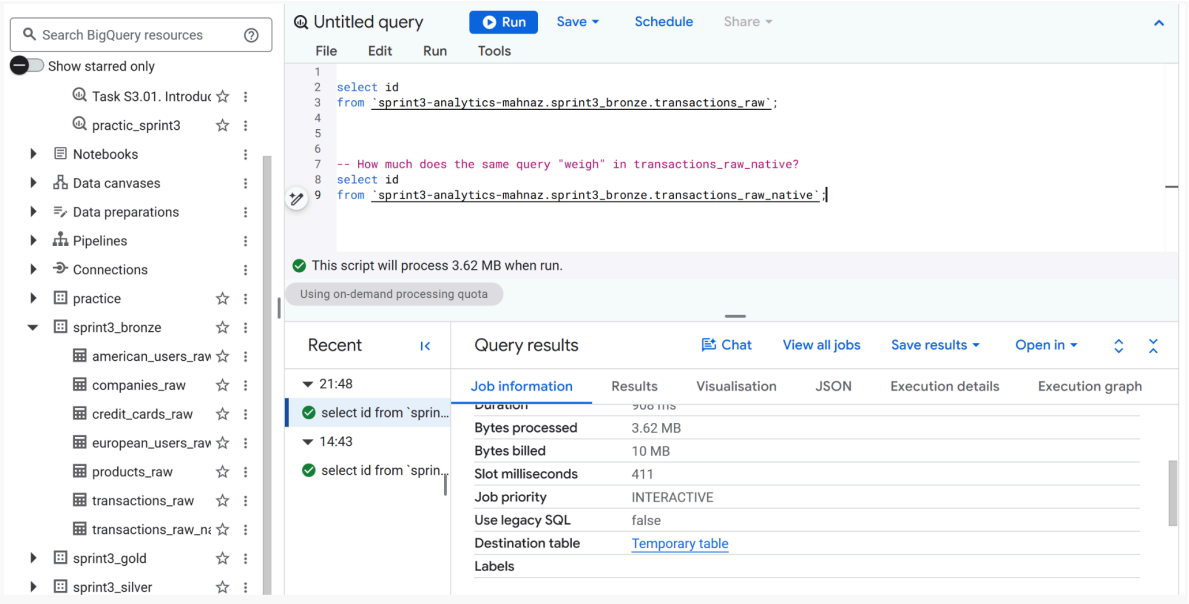
Design a test to compare the cost of reading a single column (ex: id) in the external table vs. the native table. Use Bytes Processed and Bytes Billed to make the comparison.

The native table processed less data than the external table (**3.62 MB vs. 12.61 MB**). It also billed less data (**10 MB vs. 13 MB**). Therefore, the native BigQuery table is more efficient and cheaper for this query.

» How much does the query "weigh" `transactions_raw`?
12.61 MB



How much does the same query "weigh" in `transactions_raw_native`? 3.62 MB



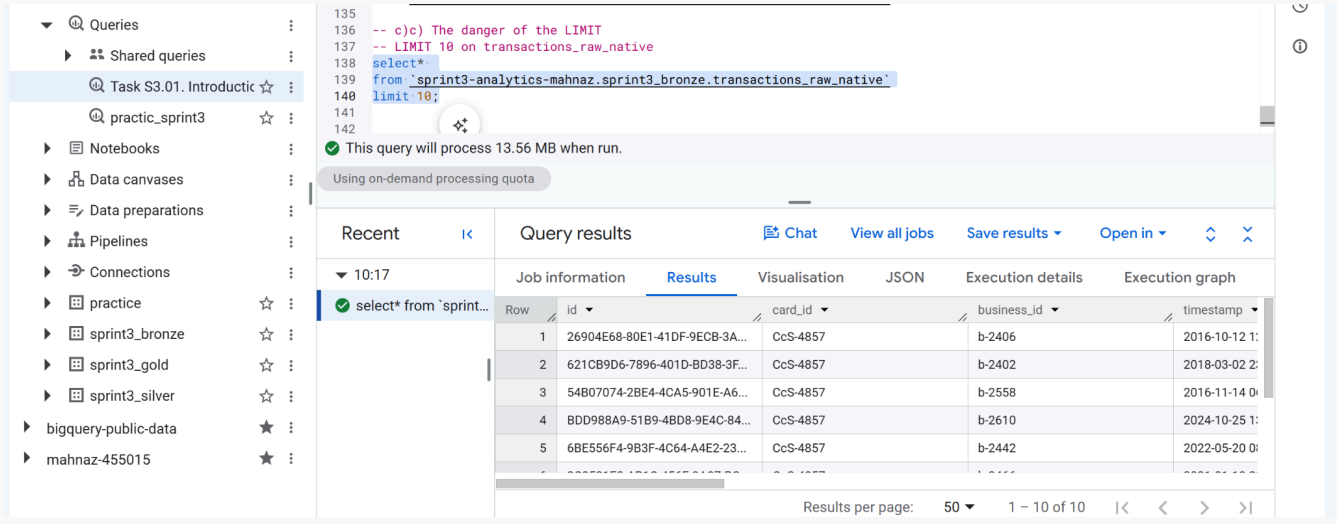
c) **The danger of the LIMIT**

Does LIMIT save processed data in BIGQUERY?

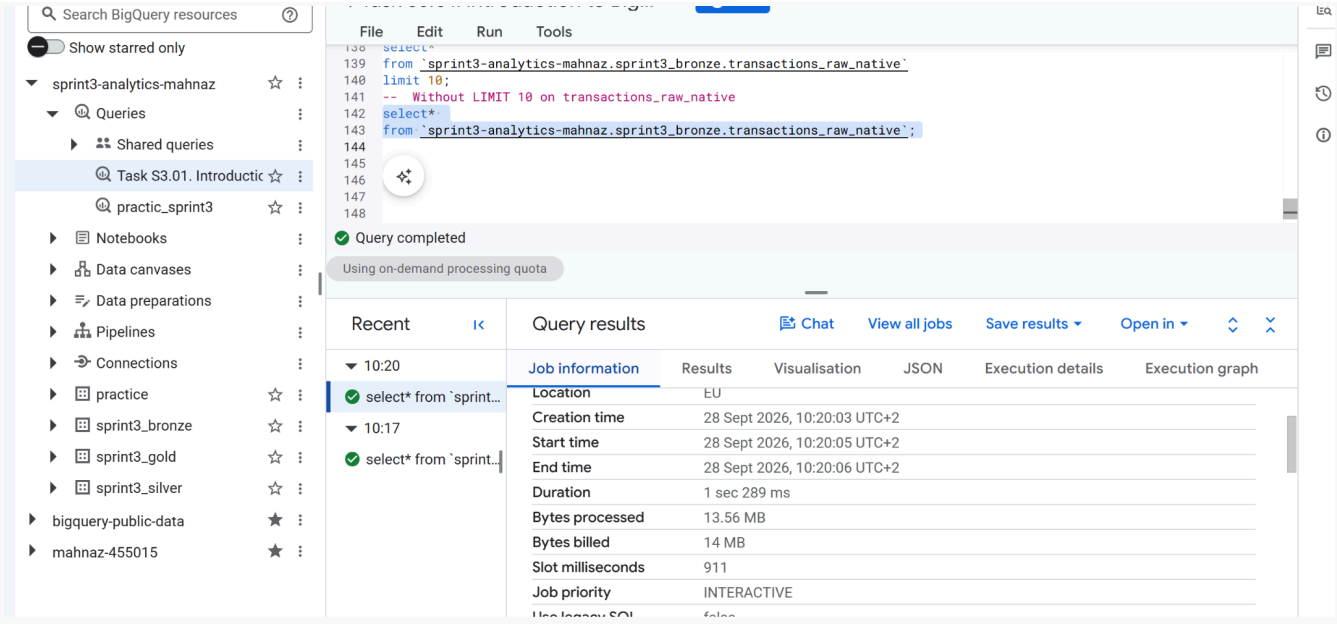
Task:

Fes un **LIMIT 10** sobre la taula transactions_raw_native y la taula transactions_raw

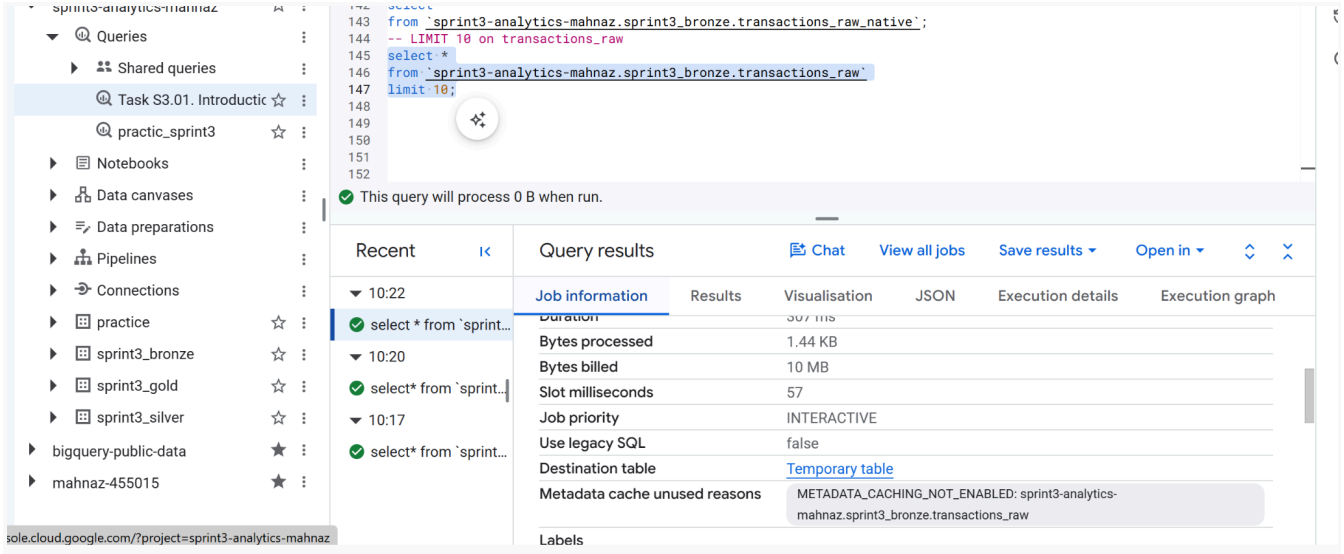
```
-- LIMIT 10 on transactions_raw_native
```



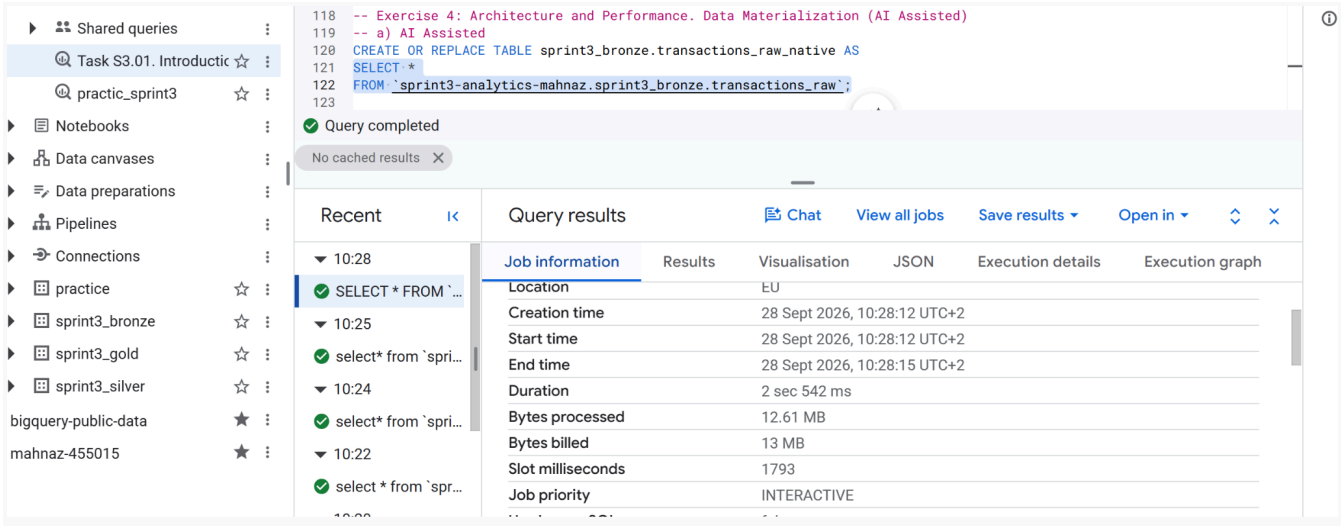

```
--  Without LIMIT 10 on transactions_raw_native
```



```
-- LIMIT 10 on transactions_raw
```




```
--  Without LIMIT 10 on transactions_raw
```

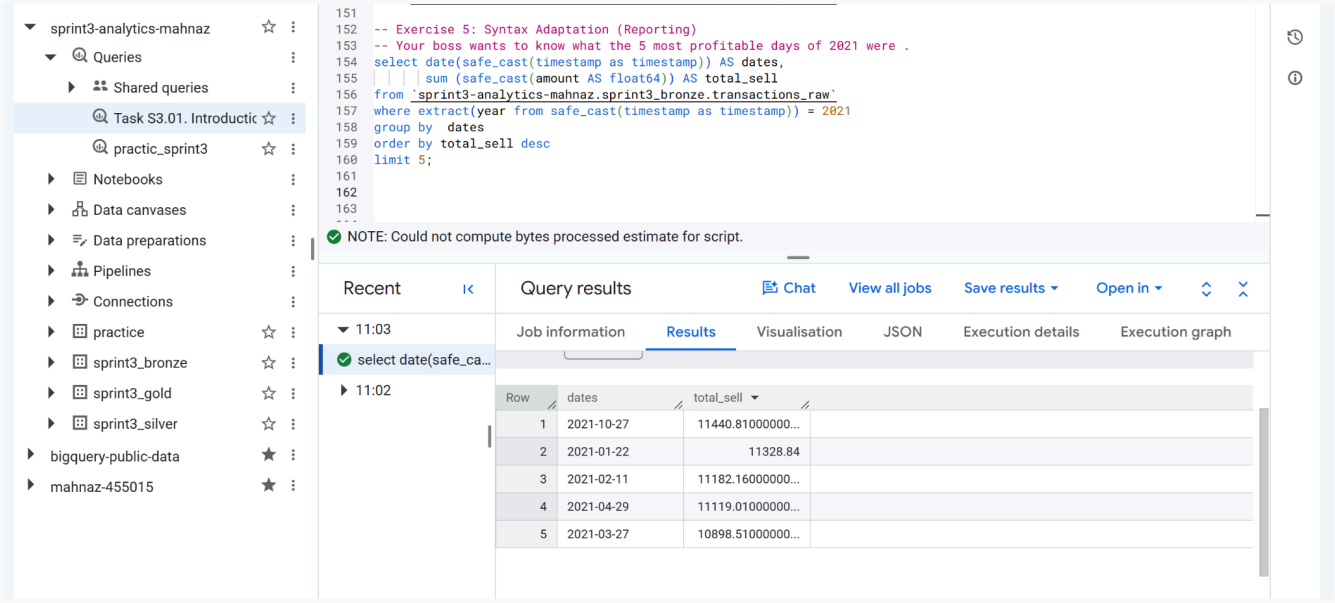


Conclusion: LIMIT does not reliably reduce query cost in BigQuery. In the native table, both queries processed 13.56 MB, with or without LIMIT 10. In the external CSV table, LIMIT 10 reduced the bytes processed in this test, but the query was still billed 10 MB. Therefore, LIMIT should be used to control the number of rows returned, not as a reliable method for reducing query cost.

Table Type	Query	Bytes Processed	Bytes Billed
Native	SELECT *	13.56 MB	14 MB
	SELECT * LIMIT 10	13.56 MB	14 MB
External	SELECT *	12.61 MB	13 MB
	SELECT * LIMIT 10	1.44 KB	10 MB

Exercise 5: Syntax Adaptation (Reporting)

Your boss wants to know what the **5 most profitable days of 2021** were .

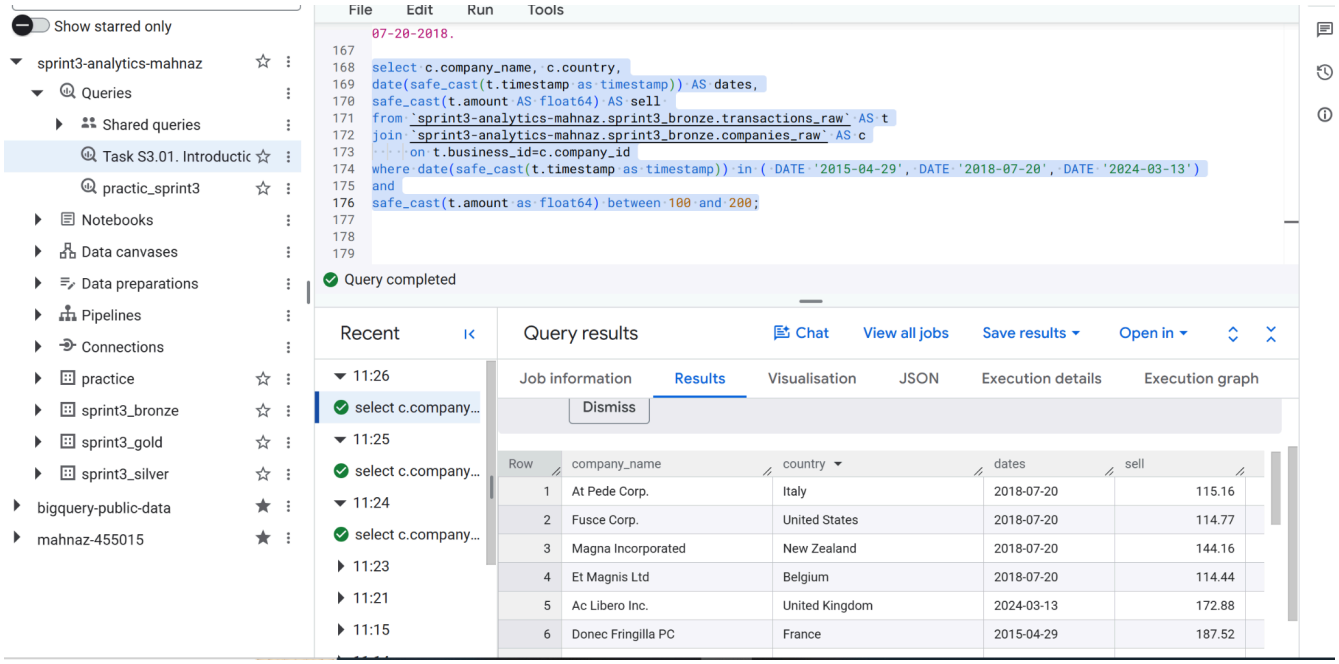


Exercise 6: Complex Queries

We need a report that crosses data.

Task:

List the name, country and date of transactions carried out by companies that made operations between 100 and 200 euros on any of these dates: 04-29-2015, 07-20-2018 or 03-13-2024.



Level 2: Cleaning and Transformation (ELT)

Exercise 1: Product Cleaning (Data Quality)

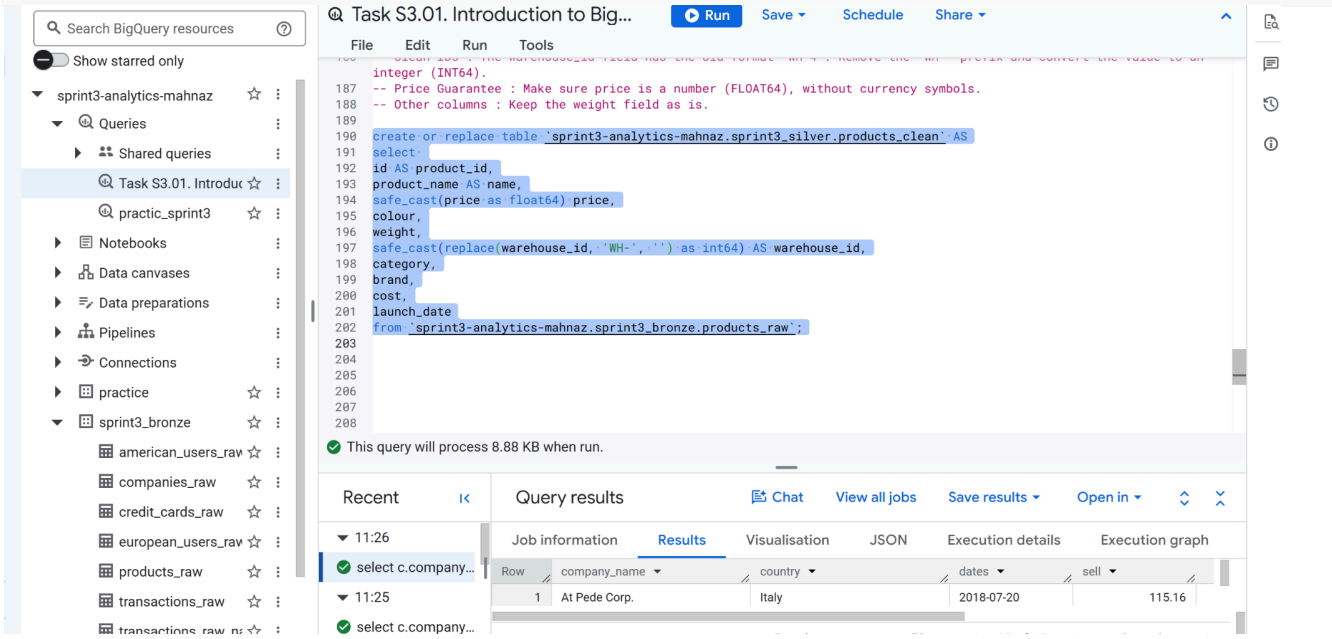
We will create the clean ("Silver") layer for products. You have the table sprint3_bronze.products_rawloaded from the CSV. The Data Governance team asks you to create a clean products table that sprint3_silver.products_cleanmeets these quality rules:

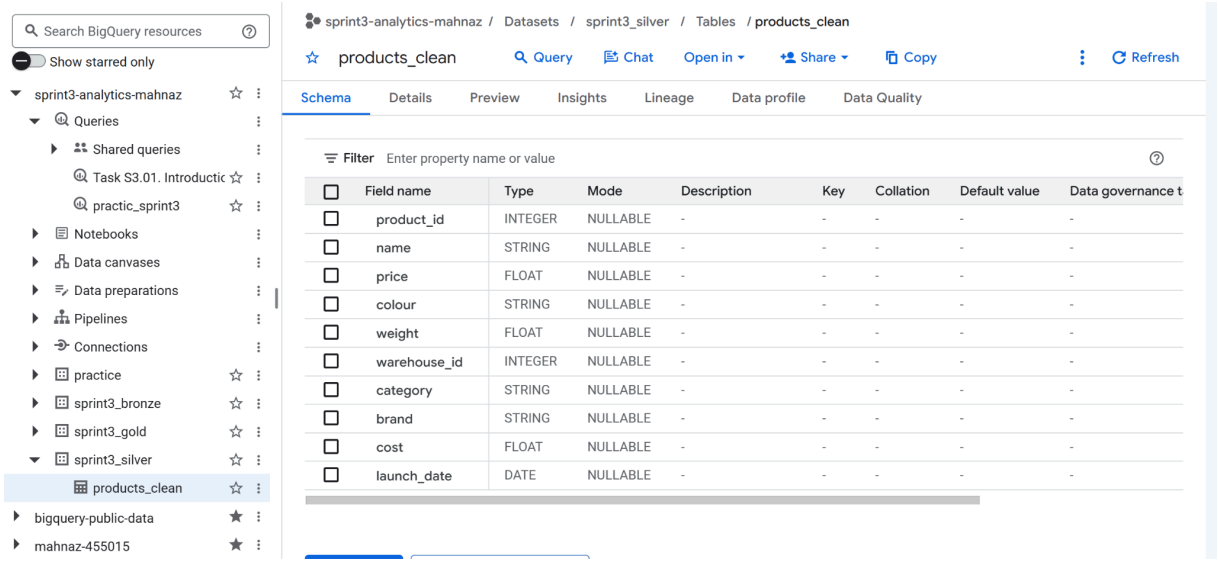
» Name Standardization : The original id column is ambiguous; rename it to product_id. Simplify the product_name column to name.

» Clean IDs : The warehouse_id field has the old format "WH-4". Remove the "WH-" prefix and convert the value to an integer (INT64).

» Price Guarantee : Make sure price is a number (FLOAT64), without currency symbols.

» Other columns : Keep the weight field as is.





Exercise 2: Creating Net Transactions (Silver Layer)

We will create the final table sprint3_silver.transactions_cleanthat guarantees the following.

» Naming Standardization : The original id column is ambiguous; rename it to transaction_id.

» Robustness in Amounts : Use SAFE_CAST on the amount field. If it fails, replace it with 0 (using IFNULL).

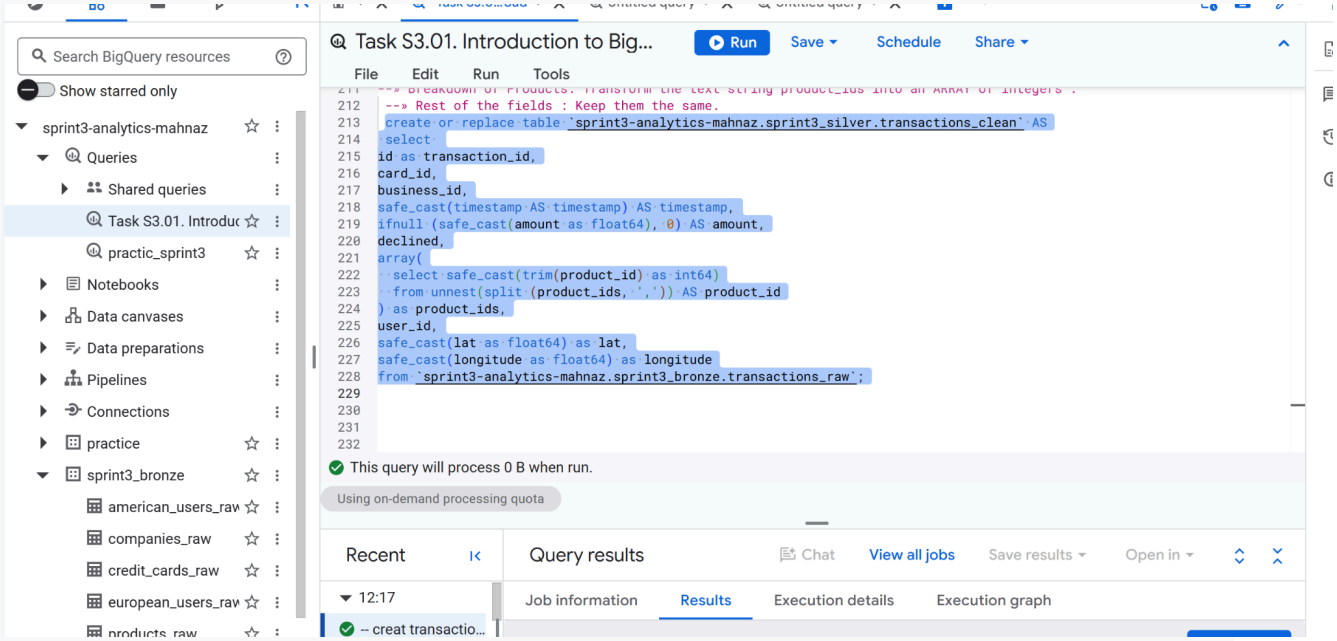
» Real Dates : Converts the timestamp field (which is STRING) to real TIMESTAMP type.

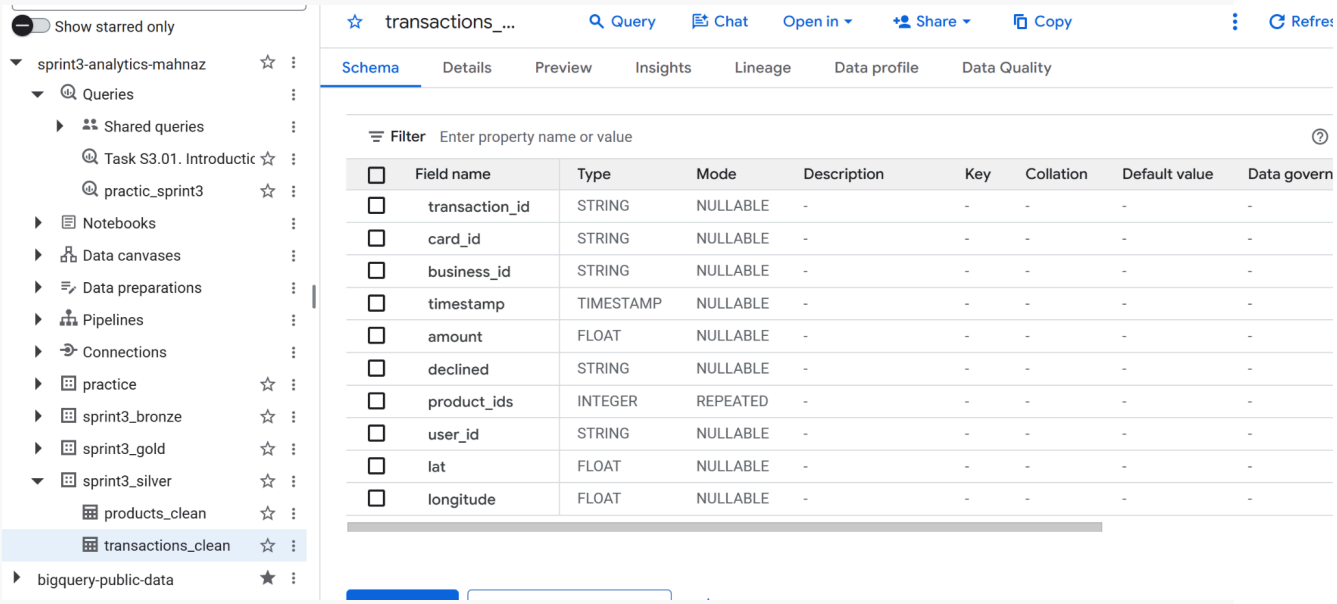
» Coordinates : Ensure that lat and longitude are FLOAT64 (use SAFE_CAST for safety).

» Breakdown of Products: Transform the text string product_ids into an ARRAY of integers .

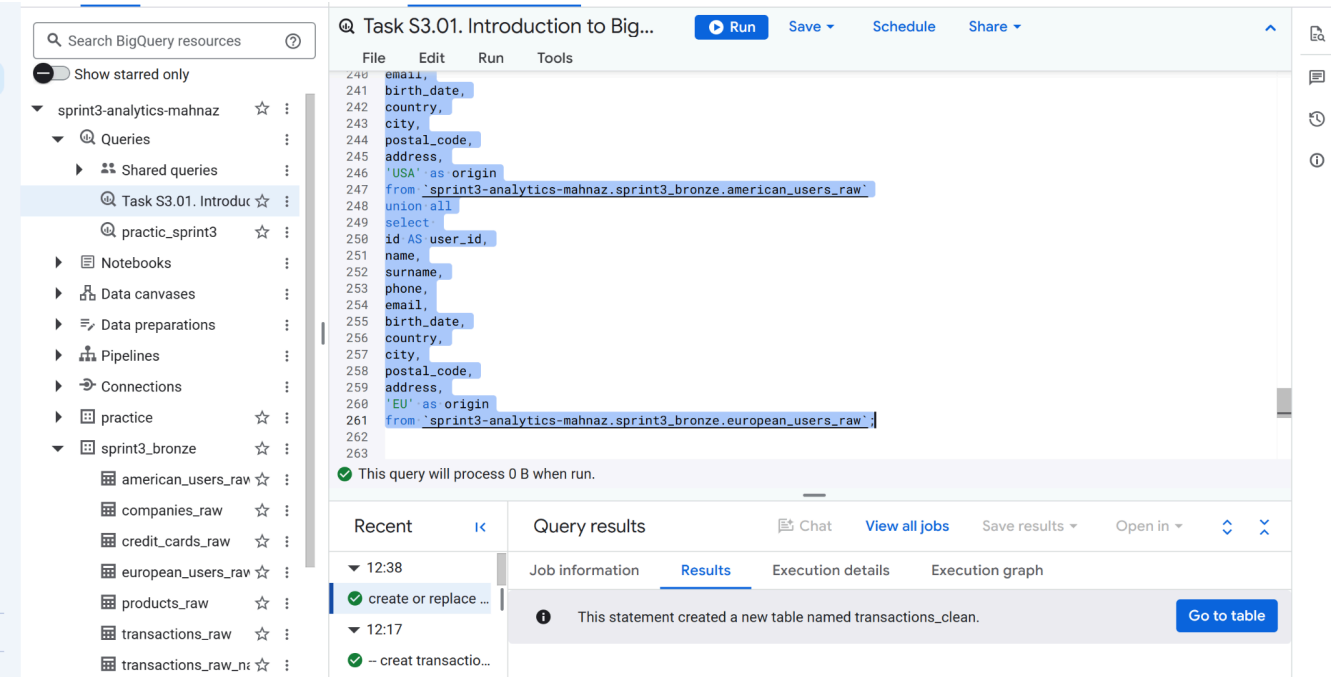
- Input: "1, 2, 3" (String)
- Output: [1, 2, 3] (Array/List of integers)

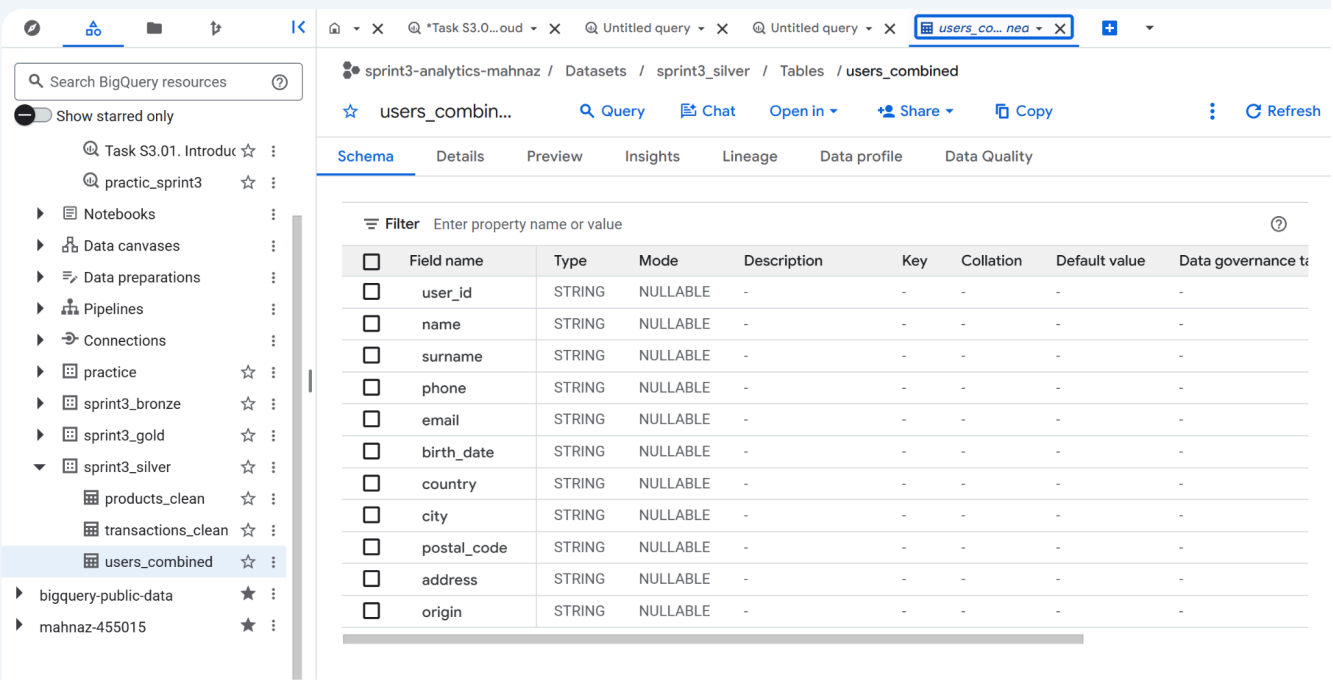
» Rest of the fields : Keep them the same.





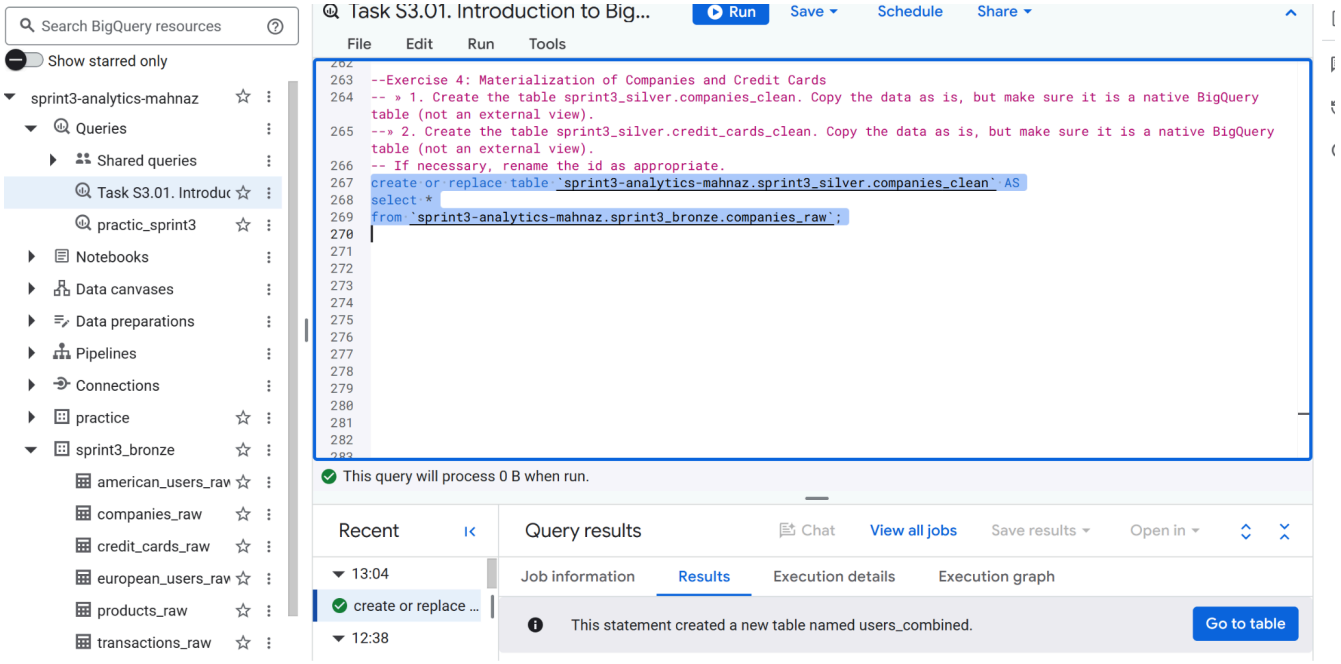
Exercise 3: User Unification (UNION)
We have users fragmented by region.
Task:
Create the table sprint3_silver.users_combined. Use UNION ALLto unify users from the US and Europe into a single master list. Add a calculated column origin to know where they come from.
» Name Standardization : The original id column is ambiguous; rename it to user_id.

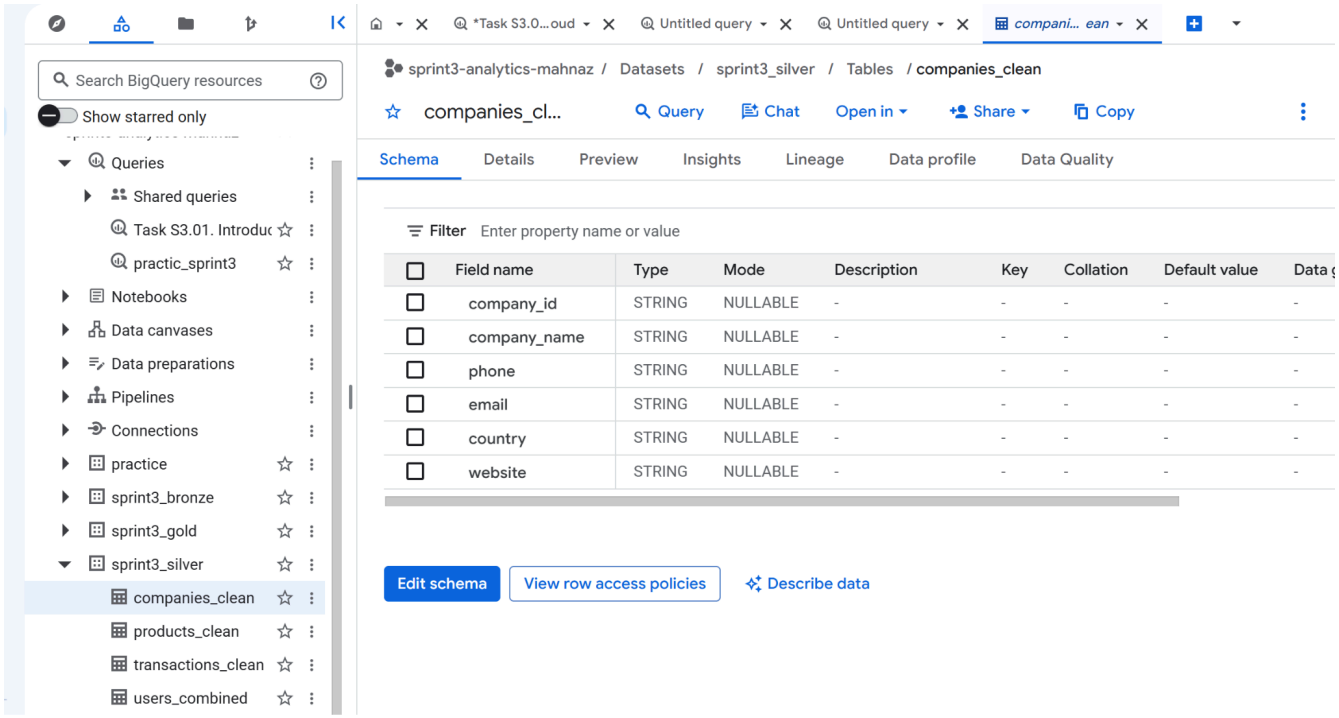




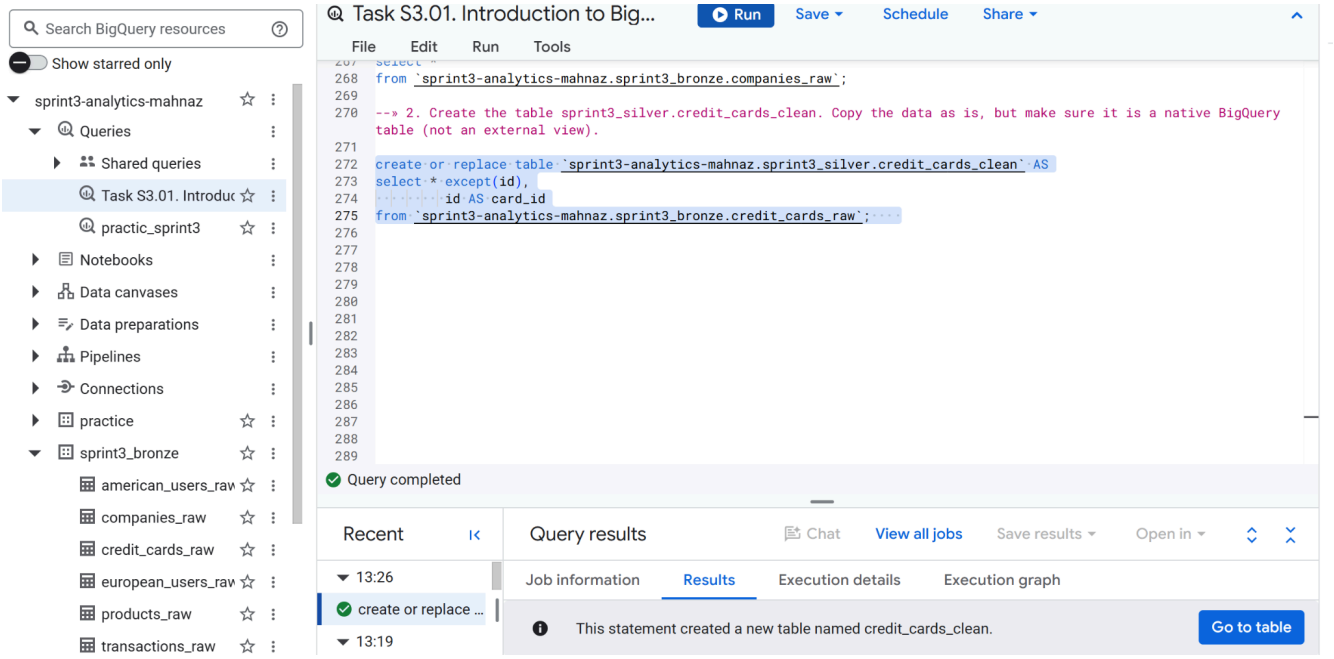
Exercise 4: Materialization of Companies and Credit Cards

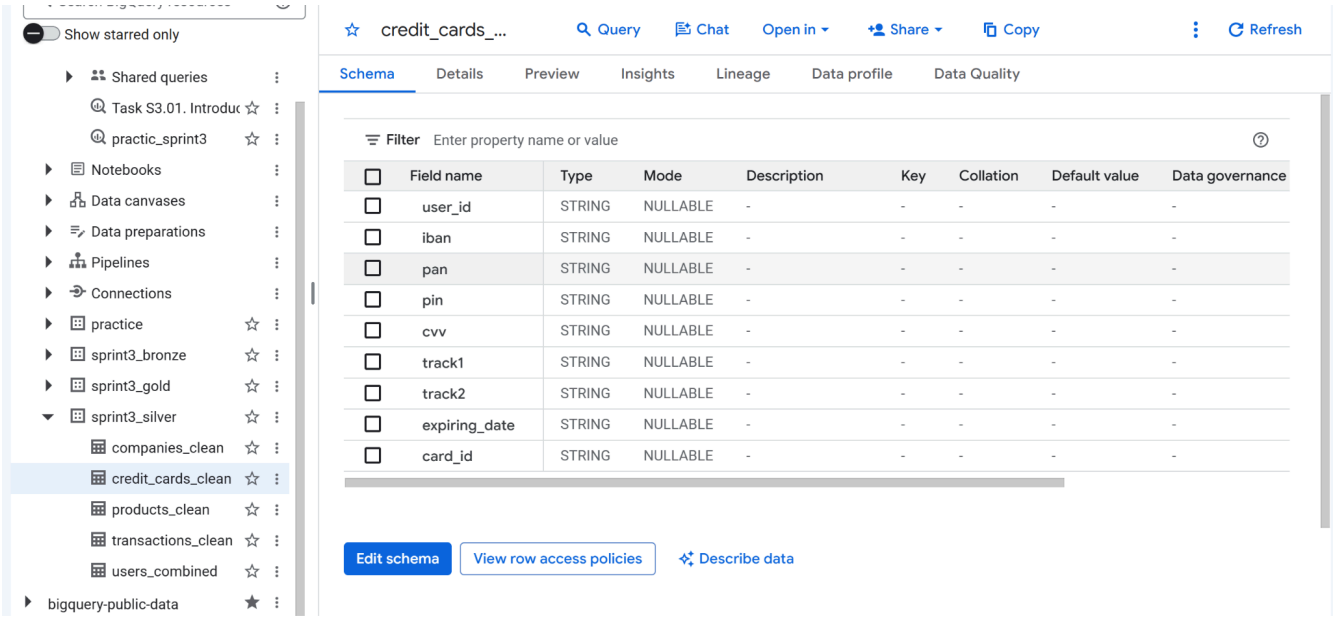
1. Create the table sprint3_silver.companies_clean. Copy the data as is, but make sure it is a native BigQuery table (not an external view).





2. Create the table sprint3_silver.credit_cards_clean. Copy the data as is, but make sure it is a native BigQuery table (not an external view).





Level 3: Data Presentation and View Creation

Exercise 1: The Marketing View (Business Logic)

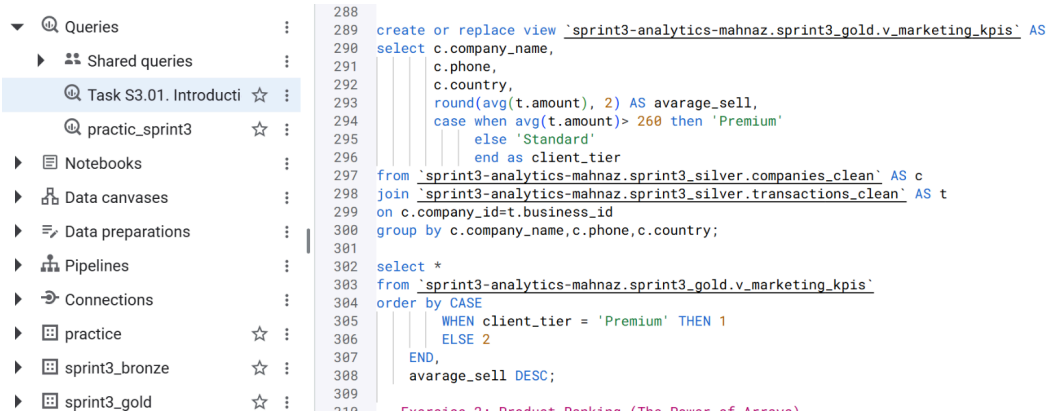
Marketing needs to segment corporate customers, but they don't know how to do JOINs. Your job is to leave them with the information ready.

🧩 Task:

Create a named view sprint3_gold.v_marketing_kpisthat displays the following information for each company:

» Company name, Phone and Country (source: companies_clean).
» Average purchase (AVG(amount) of transactions_clean).
» Customer Classification (Logic) :

- Create a calculated column called client_tier.
- If the average purchase is above €260 , label it as "Premium" .
- If equal or lower, label as "Standard" .

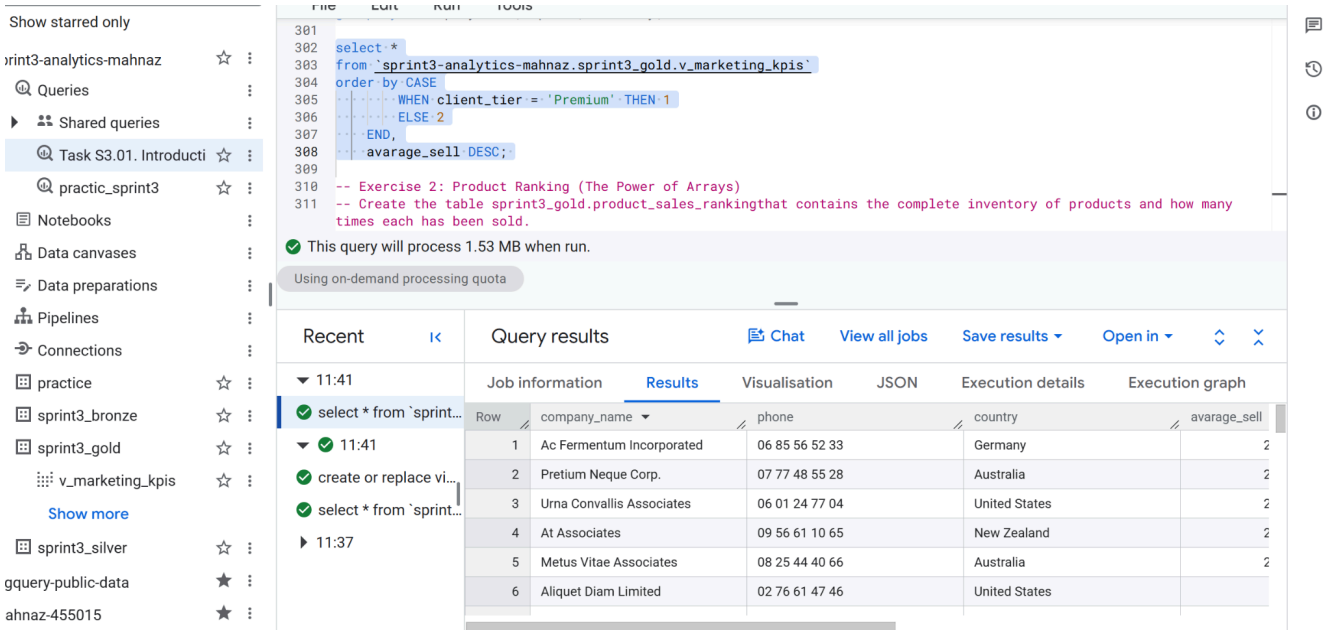


Perform a query `SELECT *` on your new view, sorting the results so that "Premium" customers appear first and, within these, those with the highest average purchase.

```sql
-- » Average purchase (AVG(amount) of transactions_clean).
-- » Customer Classification (Logic) :
-- Create a calculated column called client_tier.
-- If the average purchase is above €260 , label it as "Premium" .
-- If equal or lower, label as "Standard" .

create or replace view `sprint3-analytics-mahnaz.sprint3_gold.v_marketing_kpis` AS
select c.company_name,
       c.phone,
       c.country,
       round(avg(t.amount), 2) AS avarage_sell,
       case when avg(t.amount)> 260 then 'Premium'
            else 'Standard'
            end as client_tier
from `sprint3-analytics-mahnaz.sprint3_silver.companies_clean` AS c
join `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean` AS t
on c.company_id=t.business_id
group by c.company_name,c.phone,c.country;

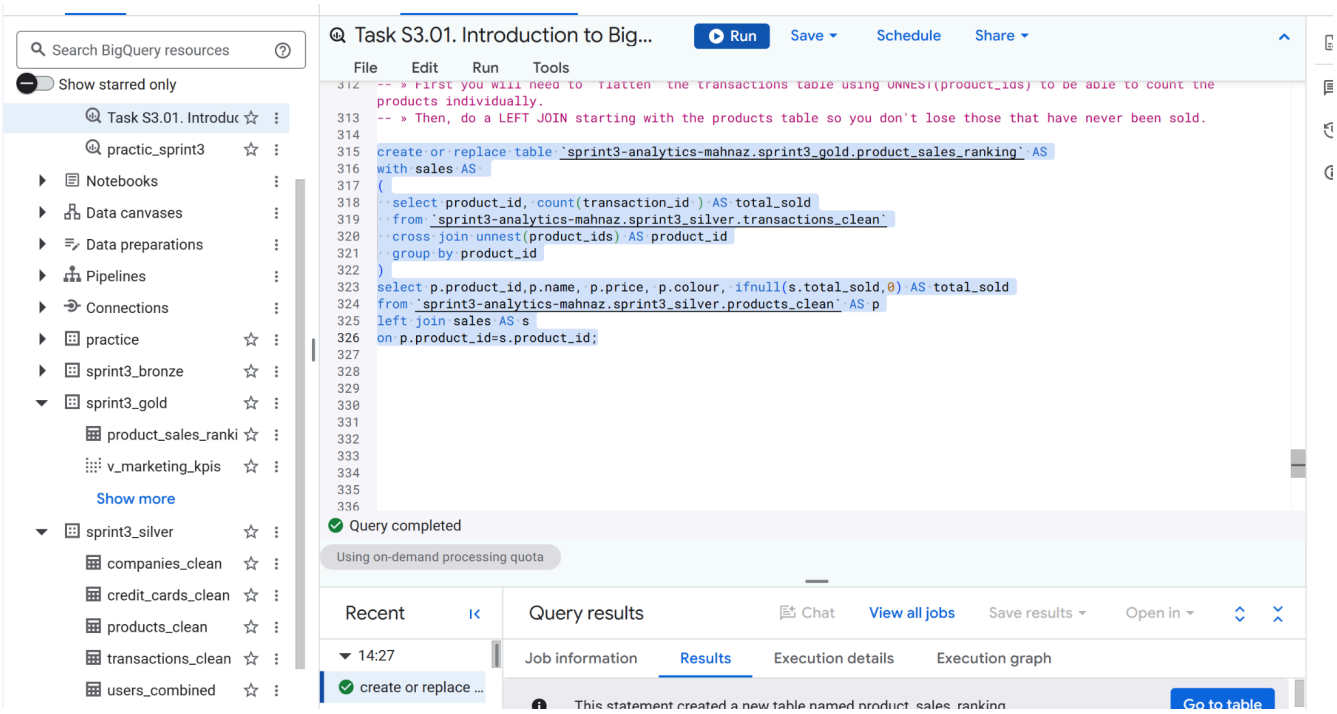
select *
from `sprint3-analytics-mahnaz.sprint3_gold.v_marketing_kpis`
order by CASE
         WHEN client_tier = 'Premium' THEN 1
         ELSE 2
    END,
    avarage_sell DESC;
```

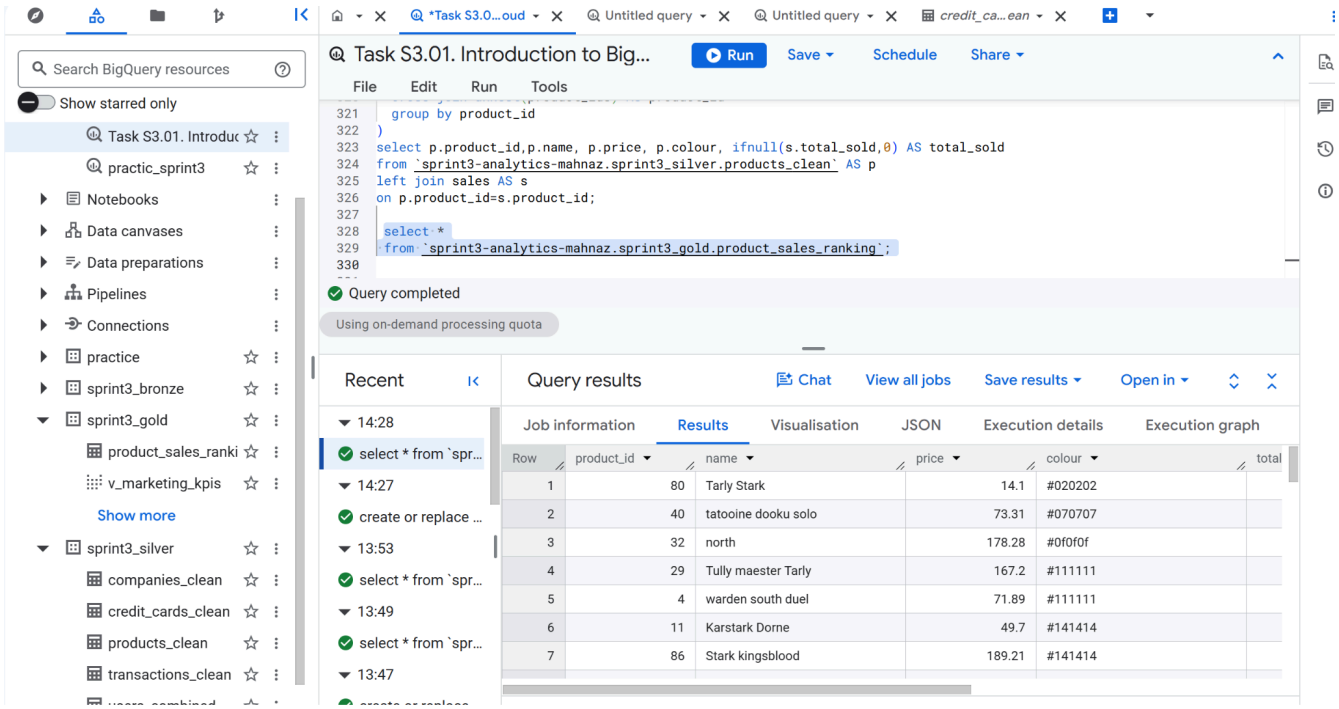


Exercise 2: Product Ranking (The Power of Arrays)

Create the table `sprint3_gold.product_sales_ranking`that contains the complete inventory of products and how many times each has been sold.

» Product Detail : Must include product_id, name, price and color (they come from the products_clean table).
» Business Metric : A new total_sold column that counts how many times this product appears in transactions.
» Integrity : All products must appear, even those with 0 sales (they may need to be discontinued).
» In transactions_clean, the products are "encapsulated" within an Array for each transaction.
» First you will need to "flatten" the transactions table using UNNEST(product_ids) to be able to count the products individually.
» Then, do a LEFT JOIN starting with the products table so you don't lose those that have never been sold.





Exercise 3: Exporting Results

Export the data from the product_sales_ranking table to Google Sheets or download the CSV file locally.

